

MWORKS 2026a

Syslab基础功能

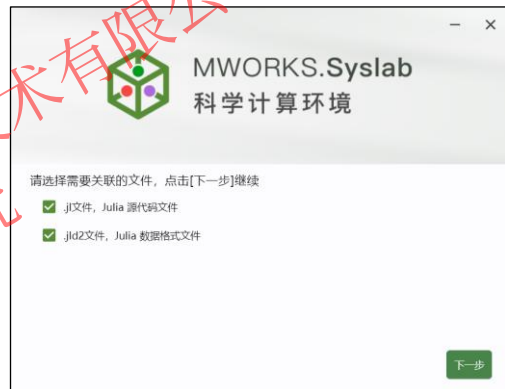
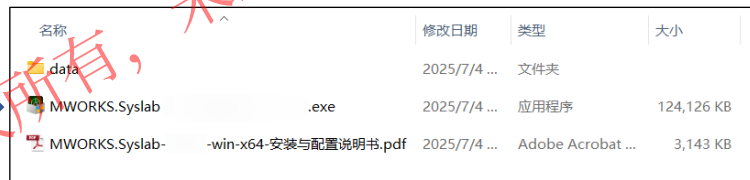


软件下载与注册



安装步骤:

MWORKS.Syslab 2026a安装包为iso光盘映像文件。其中data文件夹为相关资源文件，包括Julia仓库等；.exe文件为MWORKS.Syslab安装程序；PDF文件为安装与配置说明书。



软件下载与注册

- 师生可使用合作高校**学校邮箱**直接注册申请教育版使用许可，教育版许可每180天需要重新激活
- 个人版仅限个人非商业用途使用，且功能有所欠缺



没有账号? [立即注册](#)

账号密码登录

邮箱验证码登录

* 账号

* 密码

☒ 同意并遵守《服务协议》和《隐私政策》

忘记密码?

登录



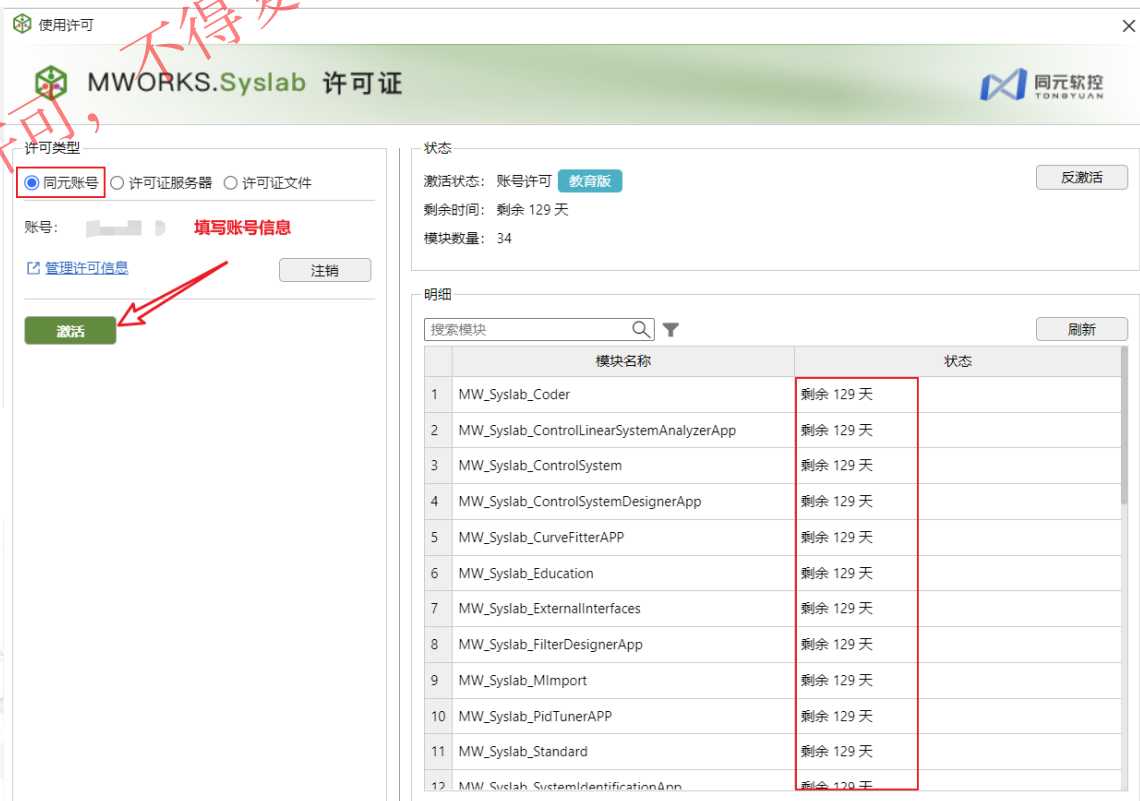
MWORKS产品

申请MWORKS使用许可

面向申请MWORKS使用许可，体验新一代科学计算与系统建模的仿真平台带来的无限可能

我的许可

产品名称	许可类型	已绑定设备	剩余时间(天)	申请日期	过期日期	当前状态
1 MWORKS.Syslab 科学计算环境	账号许可	1	129	2025-07-08	2026-01-04	激活通过
2 MWORKS.Sysplorer 系统建模仿真环境	账号许可	1	128	2025-07-07	2026-01-03	激活通过



国产科学计算与系统建模仿真赋能专项行动

MWORKS 2026

——星火计划

“星星之火，足以燎原。”

国产自主不只是一个愿景，更是一场可以由每一位工程师共同参与的实证之旅。为加速这一进程，同元软控正式启动MWORKS 免费升级与试用活动，期待联合广大用户、生态伙伴，在真实场景中打磨国产工业软件实战性能。

星火已燃，静待同路人。我们诚邀您加入“MWORKS星火计划”，共同见证并推动中国工业软件的自强历程！

★ 活动详情：请关注“同元软控” / “MWORKS” 官方公众号

★ 2026年2月7日-12月31日，为企业用户提供以下专属权益：



零成本试用

全功能版本免费开放



升级支持

老用户免费版本升级



线上培训

免费参加线上培训



产品资料包

全套产品资料包



技术支持

线上/线下技术支持

国产科学计算与系统建模仿真赋能专项行动

MWORKS 2026

——星火计划

“星星之火，足以燎原。”

国产自主不只是一个愿景，更是一场可以由每一位工程师共同参与的实证之旅。为加速这一进程，同元软控正式启动MWORKS 免费升级与试用活动，期待联合广大用户、生态伙伴，在真实场景中打磨国产工业软件实战性能。

星火已燃，静待同路人。我们诚邀您加入“MWORKS星火计划”，共同见证并推动中国工业软件的自强历程！

★ 活动详情：请关注“同元软控” / “MWORKS” 官方公众号

★ 面向高校用户，长期提供以下权益



校级免费授权
全校教育版使用



课程教材评优
奖金/资源/荣誉



数智英才培养
对接企业优先就业



高影响力赛事
荣誉/奖金/就业



应用赋能培训
季度性专业辅导



百行百业共建
拉通产业产品共创



产品资料包
全套产品资料包



技术支持
线上/线下技术支持

工业知识互联平台MoHub

MoHub：设置技术交流/高校专区，提供大量常见问题FAQ，安排专业工程师解答

MoHub

搜索

MoHub首页: www.mohub.net

The screenshot displays the MoHub website interface. At the top, there is a navigation bar with the MoHub logo and links for 首页 (Home), 技术交流 (Technical Exchange), 众创共享 (Mass Creation and Sharing), 数字教学 (Digital Teaching), 创新实践 (Innovation Practice), and 高校专区 (University Special Zone). A search bar is located on the right side of the navigation bar. Below the navigation bar, the main content area is divided into several sections. On the left, there is a sidebar with a '专栏' (Column) section listing various topics like Syslab基础平台, Julia语言, 工具箱, Sysplorer基础平台, AI for MWORKS, Modelica语言, 模型库, NUMAP, and 其他. Below this is a '标签' (Tag) section with tags like 安装激活, 系统建模, 科学计算, AI, 控制工程, 工业机器人, 船舶, 汽车, 核能, 航空航天, and 展开更多. The main content area features a '问题搜索' (Problem Search) bar and a '最新' (Latest) section. A prominent orange banner reads '官方配套课程资料现已正式上线!' (Official accompanying course materials are now officially online!). Below this, there are several posts, including one titled '方程数大于300应该怎么办?' (What should I do if the number of equations is greater than 300?) and another titled '这个模型库在哪?' (Where is this model library?). On the right side of the main content area, there is a '资料中心' (Resource Center) section with a list of resources, including '官网培训课程配套资料', '模型库手册及配套案例', '智能装备系统模型集成—2024蓝...', '智能装备控制算法开发—2024蓝...', 'MWORKS课程资料-预览版', '智能装备数字样机构建—2024蓝...', 'MWORKS.Sysplorer用户手册', and 'MWORKS.Syslab用户手册'. Below the resource center is a 'FAQ' section with a list of frequently asked questions, such as '# Modelica如何调用Refprop', '# 模型连线时提示“连接器类型不匹...', '# Sysblock问题合集', '# “组件的类型”***查找不到, 组件全...', '# 使用Fromworkspace时, 出现“Erro...', '# 出现“Error:divide-zero error”问题...', '# 代码出现“DimensionMismatch: di...', '# Julia代码中变量已经初始化定义, ...', and '# 电学模型仿真出现方程奇异错误怎...'. At the bottom right of the interface, there is a '客服' (Customer Service) button.

客服

CONTENTS

目录 ➔



01 MWORKS.Syslab 2026a 软件概览



02 软件基础设置



03 脚本构建



04 代码调试



05 结果查看



06 入门实践：太阳能电池板性能验证

PART 01 ➔

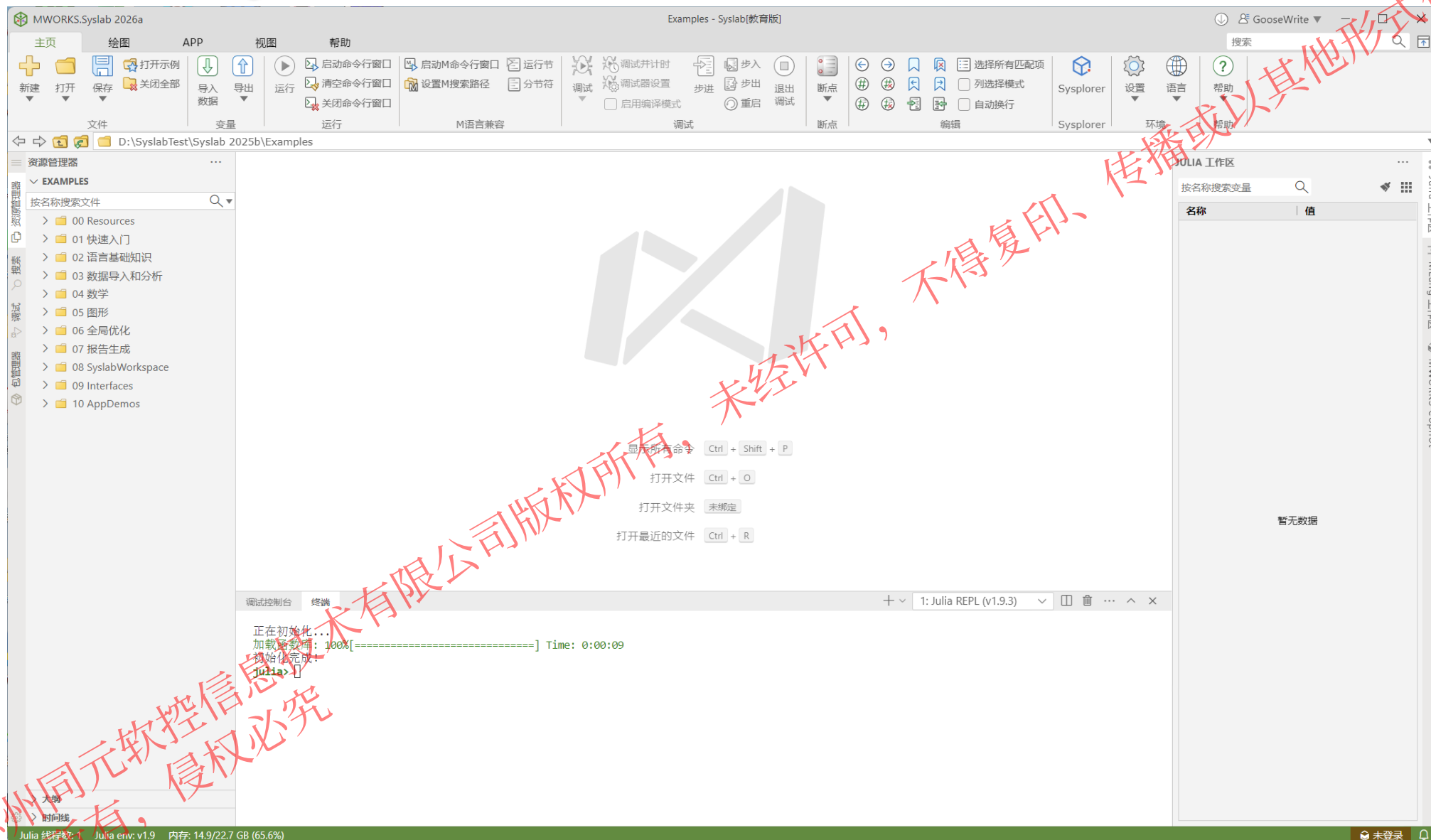
MWORKS.Syslab 2026a

软件概览

01

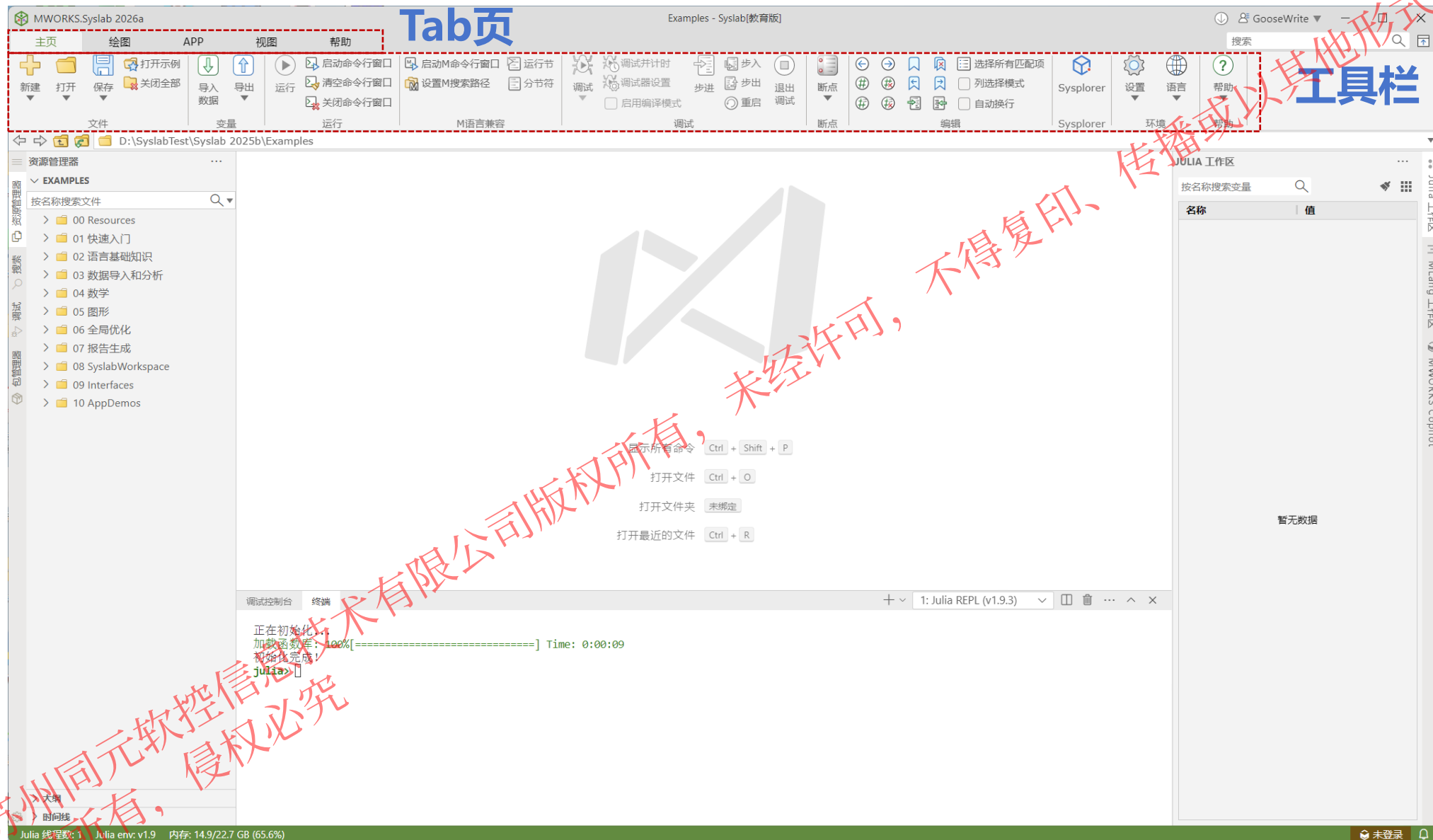
@苏州同元软件信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

1. MWORKS.Syslab 2026a 软件概览

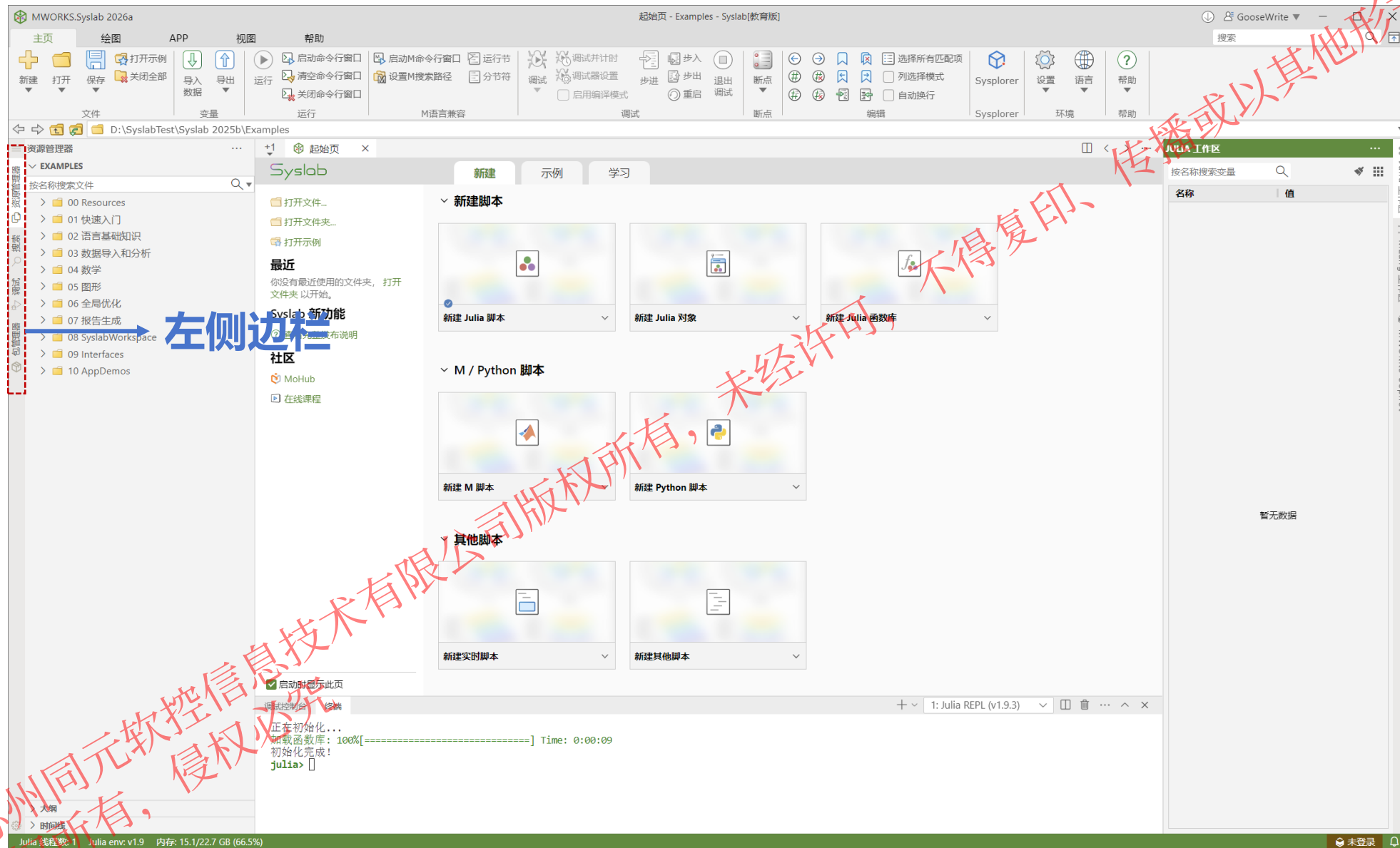


@苏州同元软控信息技术有限公司版权所有，侵权必究

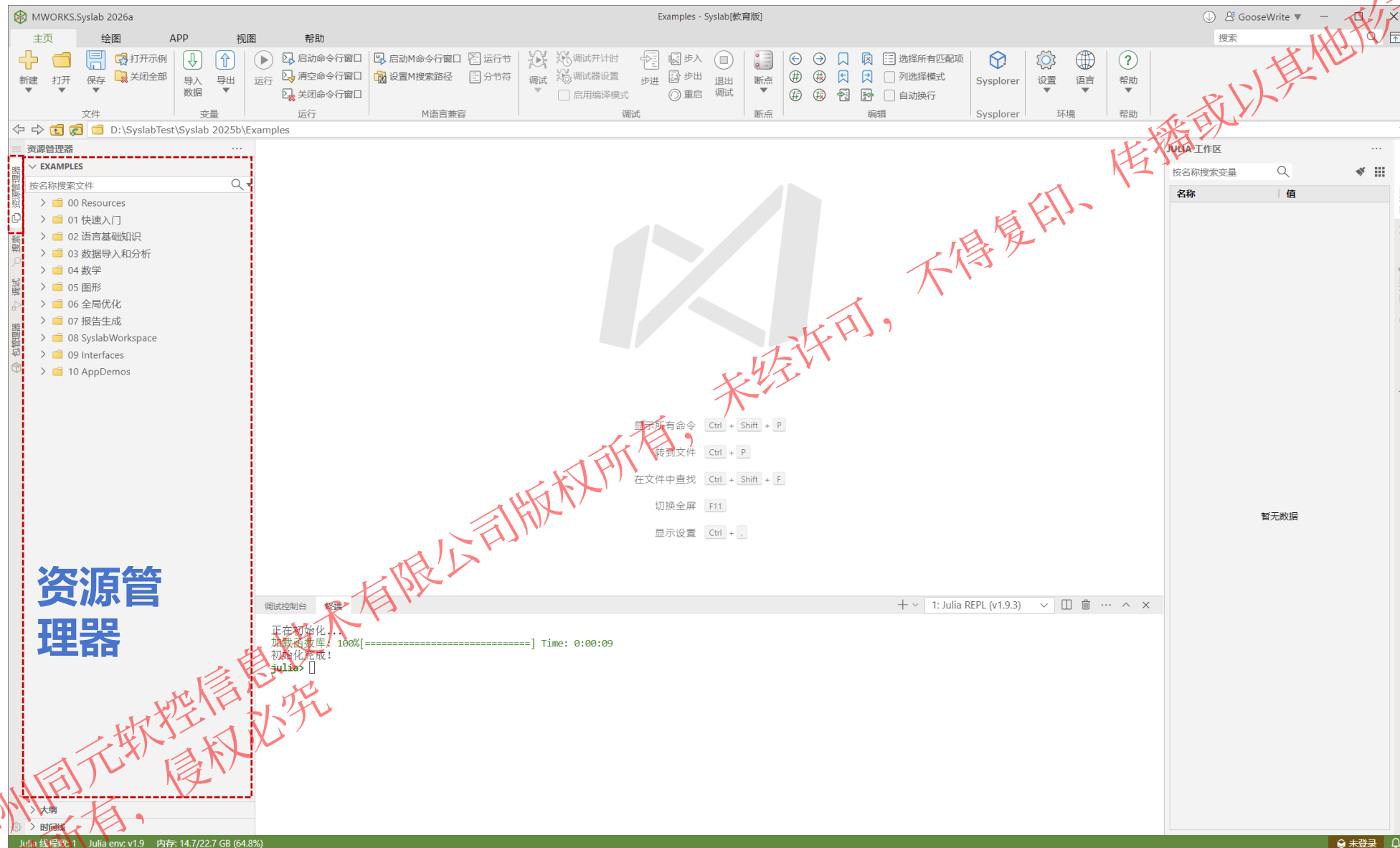
1. MWORKS.Syslab 2026a 软件概览



1. MWORKS.Syslab 2026a 软件概览



1. MWORKS.Syslab 2026a 软件概览

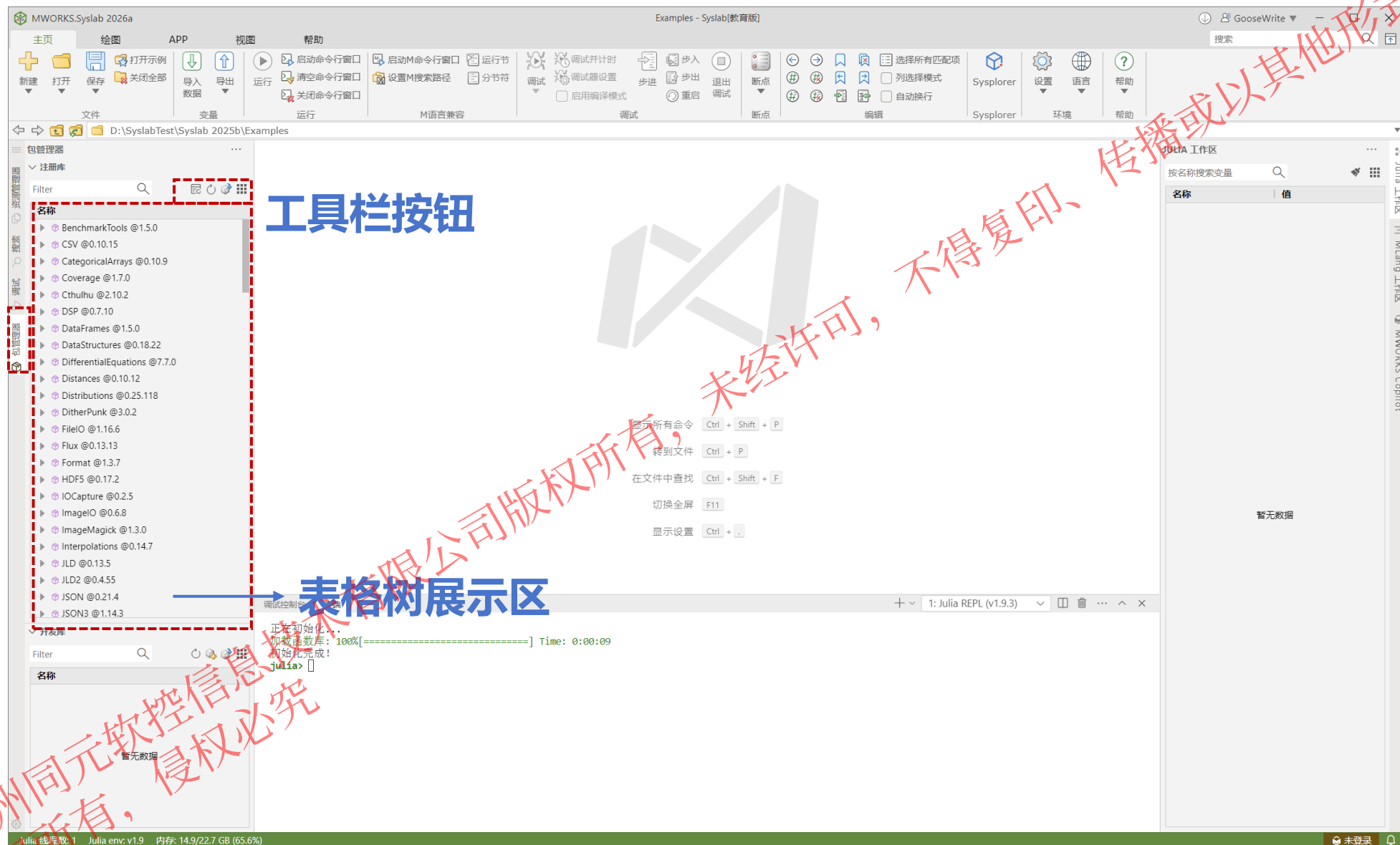


资源管理器

正在初始化...
加载函数库: 100%[=====] Time: 0:00:09
初始化完成!
julia>

@苏州同元软控信息技术有限公司版权所有, 侵权必究

1. MWORKS.Syslab 2026a 软件概览



传播或以其他形式使用

未经许可，不得复印、

@苏州同元软控信息技术有限公司
版权所有，侵权必究

1. MWORKS.Syslab 2026a 软件概览

Figure 1

Solution of $y' = -2y + 2 \cos(t) \sin(2t)$

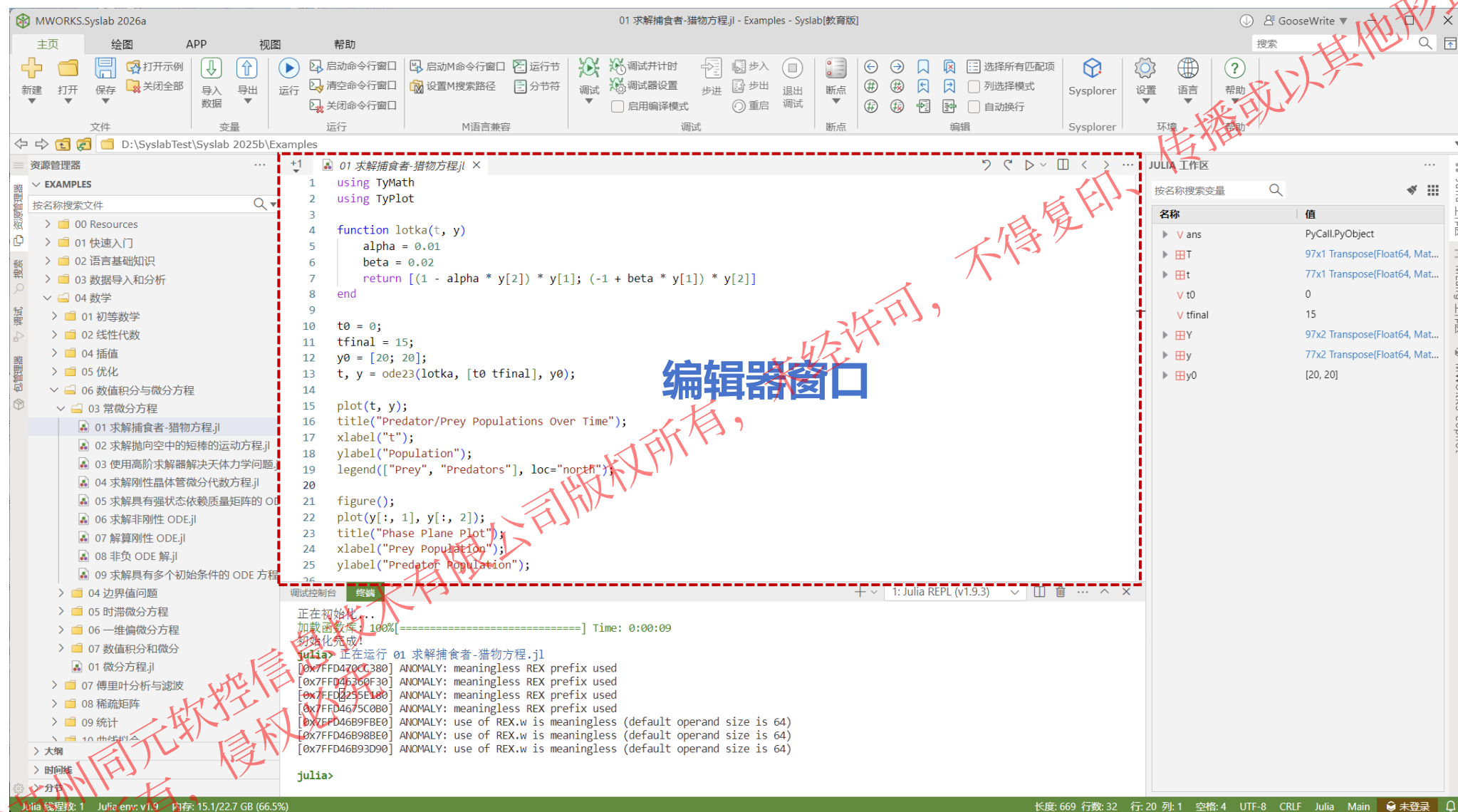
```
1 using TyMath
2 using TyPlot
3
4 # 定义ODE函数
5 yprime = (t, y) -> -2*y + 2*cos(t)*sin(2*t)
6
7 # 时间区间和初始条件
8 tspan = [0 3]
9 y0 = 0
10
11 # 求解ODE
12 t, y, = ode45(yprime, tspan, y0)
13
14 # 绘制结果
15 plot(t, y, "-o")
16 xlabel("t")
17 ylabel("y")
18 title("Solution of y' = -2y + 2 cos(t) sin(2t)")
```

正在初始化...
初始化完成!
julia>

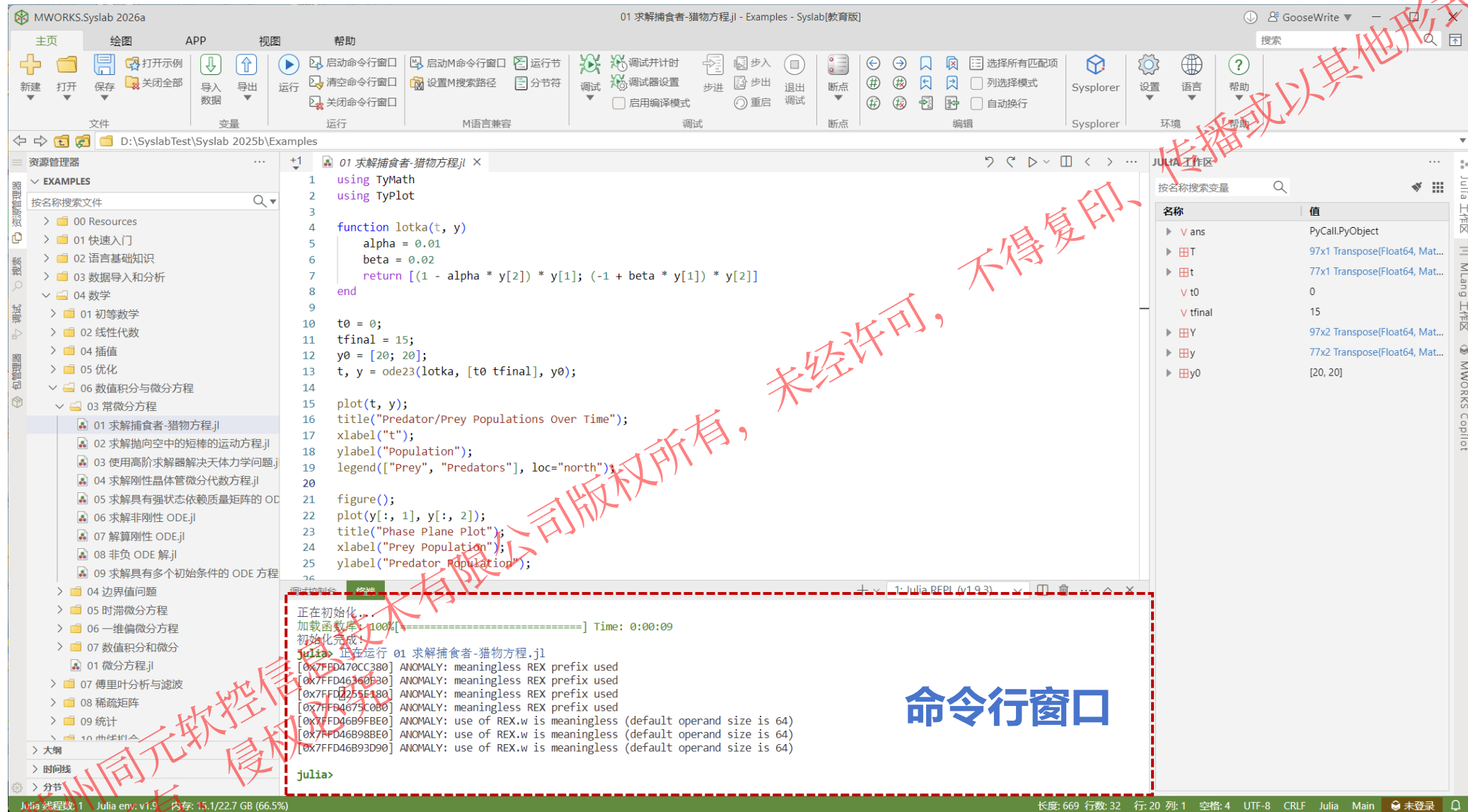
长度: 271 行数: 18 行: 18 列: 49 空格: 4 UTF-8 CRLF Julia Main Copilot

行: 8, 列: 34 空格: 4 UTF-8 CRLF Julia Main Copilot

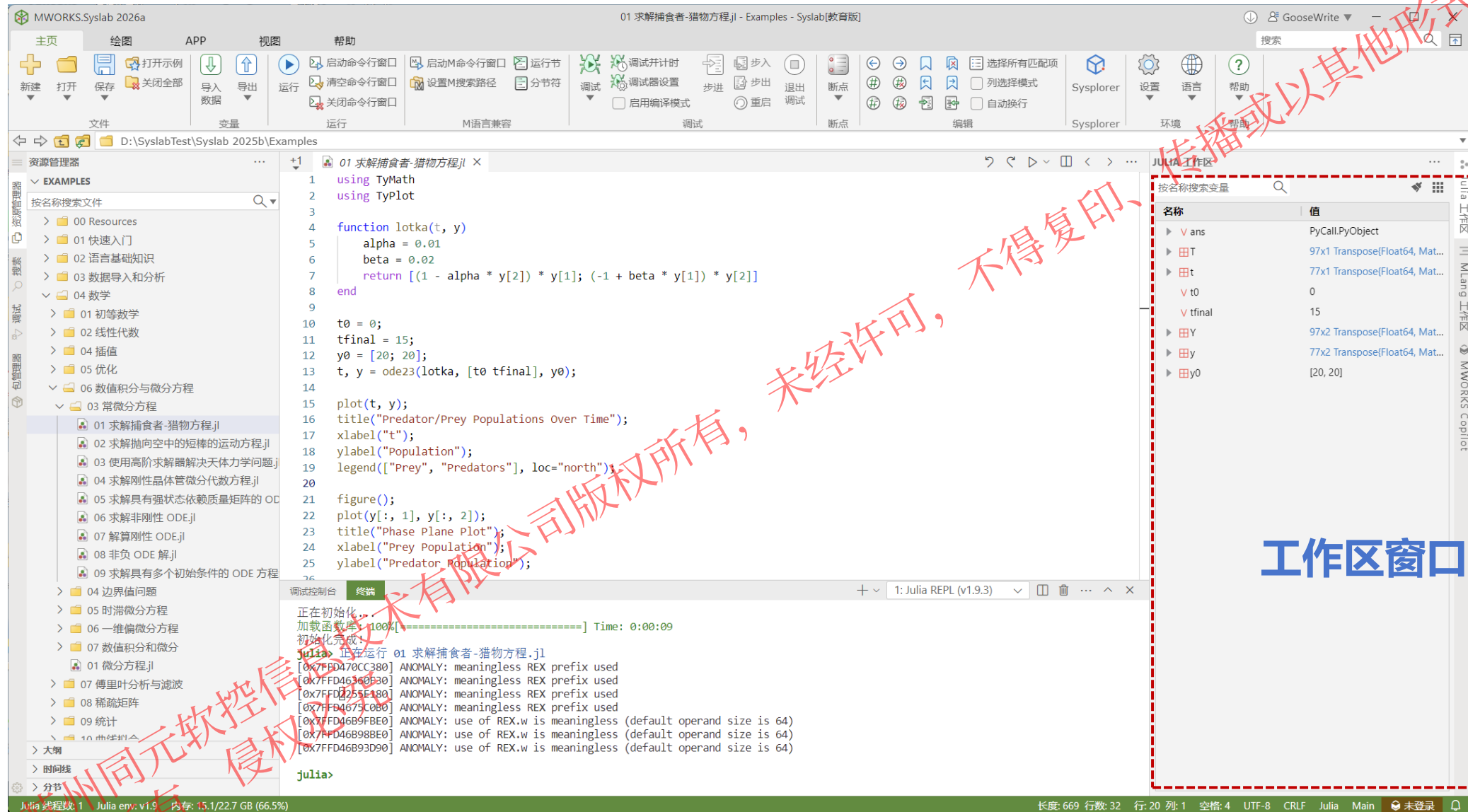
1. MWORKS.Syslab 2026a 软件概览



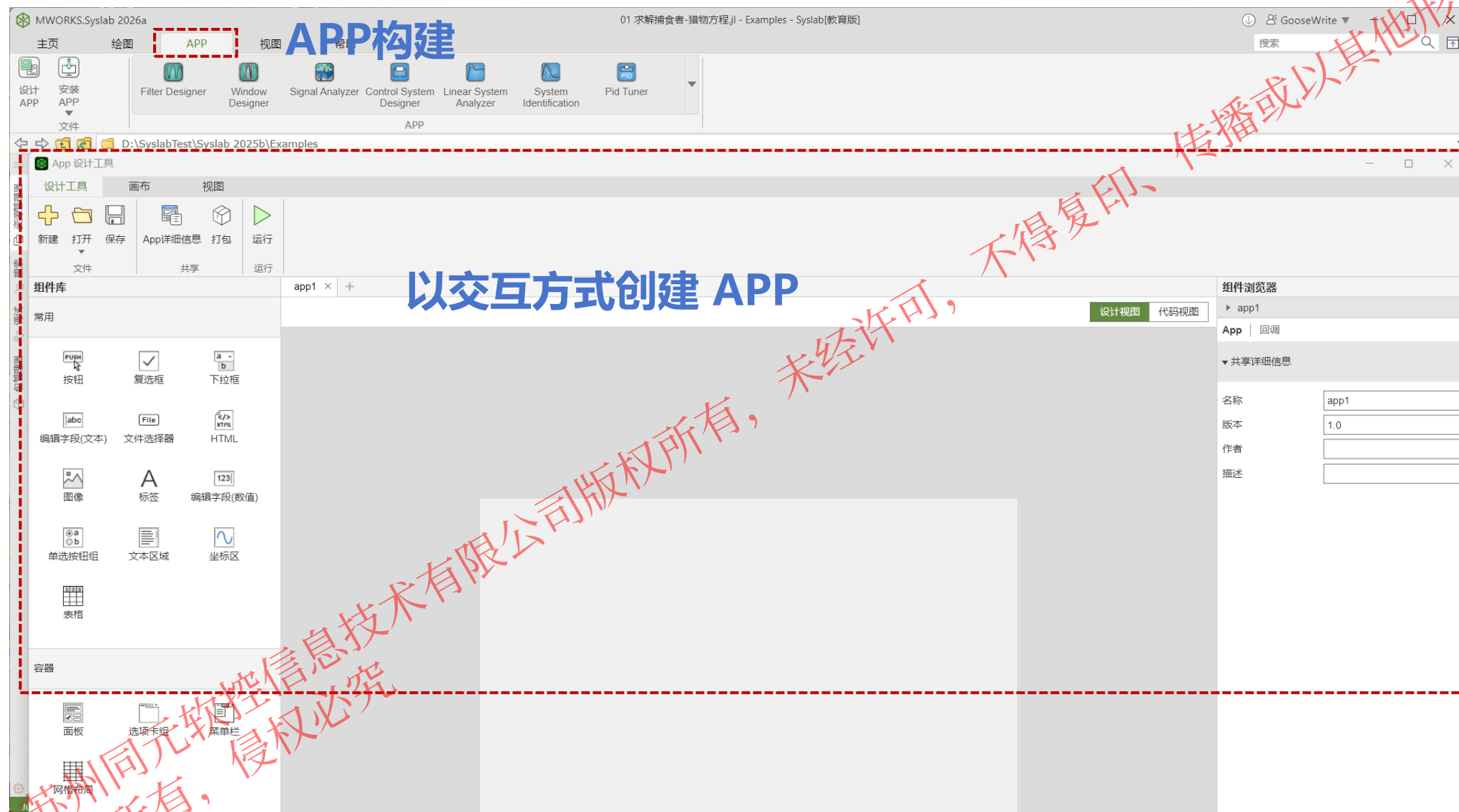
1. MWORKS.Syslab 2026a 软件概览



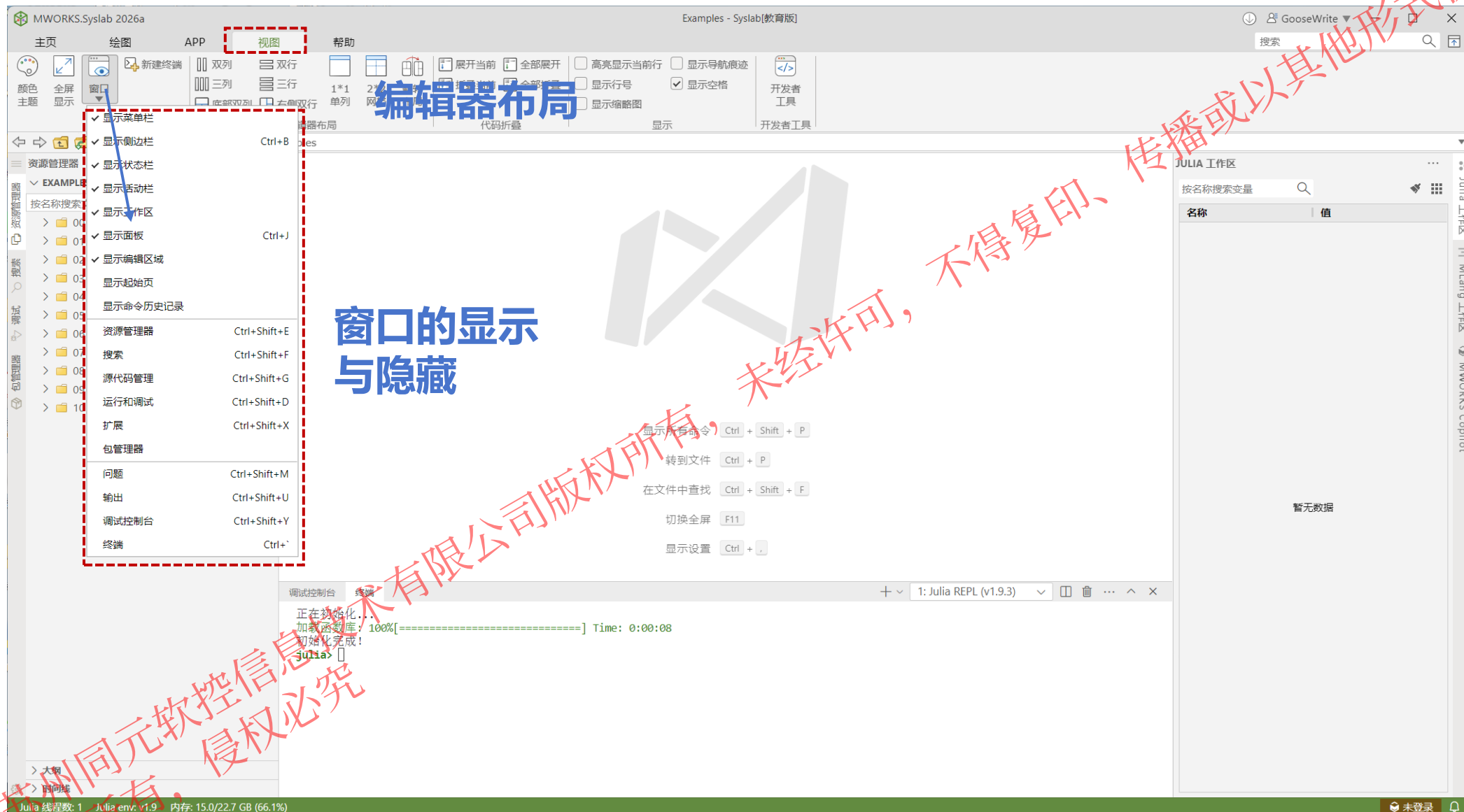
1. MWORKS.Syslab 2026a 软件概览



1. MWORKS.Syslab 2026a 软件概览



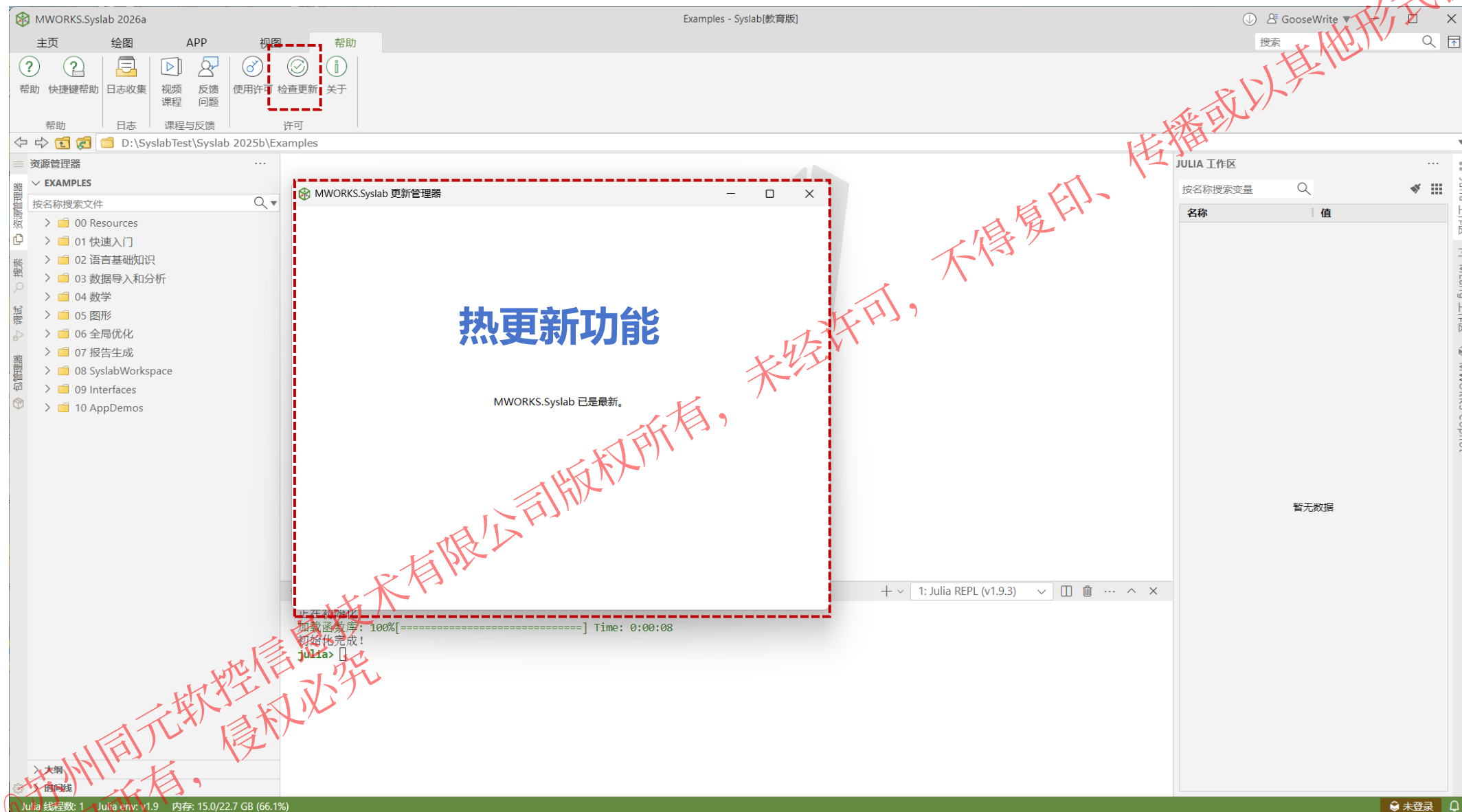
1. MWORKS.Syslab 2026a 软件概览



未经许可，不得复印、传播或以其他形式使用

同元软控信息技术有限公司版权所有，侵权必究

1. MWORKS.Syslab 2026a 软件概览



1. MWORKS.Syslab 2026a 软件概览

视频课程：提供Syslab软件使用视频课程



02

PART 02 ➔

软件基础设置

@苏州同元软控信息技术有限公司版权所有，侵权必究

未经许可，不得复印、传播或以其他方式使用

2.1 工具箱加载

点击主页菜单栏中的  首选项，设置**预加载**同元商业工具箱，每次软件启动自动加载函数库



预加载

☒ 控制启动命令行窗口时是否预加载函数库

高级配置

- 基础
 - ☒ 基础库(TyBase)
 - ☒ 数学库(TyMath)
 - ☒ 图形库(TyPlot)
 - ☐ 图像库(TyImages)
 - ☐ 地理图库(TyGeoGraphics)
 - ☐ 报告生成库(TyReportGenerator)
- 数学、统计和优化
 - ☐ 统计库(TyStatistics)
 - ☐ 曲线拟合库(TyCurveFitting)
 - ☐ 符号计算库(TySymbolicMath)
 - ☐ 优化库(TyOptimization)
 - ☐ 全局优化库(TyGlobalOptimization)
- 控制系统
 - ☐ 控制系统库(TyControlSystems)
 - ☐ 系统辨识库(TySystemIdentification)
 - ☐ 鲁棒控制库(TyRobustControl)
- 信号处理和无线通信

Syslab 2025 默认加载3个函数库：

- 基础库 (TyBase)
- 数学库 (TyMath)
- 图像库 (TyPlot)

勾选预加载后，需重启命令行窗口：

1. 关闭命令行窗口
2. 启动命令行窗口

2.1 工具箱加载

在Julia中通过using使用已正确安装的工具箱，可直接调用工具箱函数

```
using TyMathCore
using MatrixPencils
using MLDatasets: MNIST
using Base.Iterators: partition
using Printf, BSON
using Parameters: @with_kw
using CUDA
```

```
model = sum(A) #直接调用已预加载工具箱中的函数sum对数组A求和
```

```
julia> spzeros
ERROR: UndefVarError: `spzeros` not defined
#当多个库中的函数产生冲突时，需要使用库名+函数名的方式
julia> MatrixPencils.spzeros
spzeros (generic function with 1 method)
```

TyMathCore和MatrixPencils库中均有spzeros函数

2.2 工具箱安装和卸载

在Syslab中可以通过包管理器，管理注册库与开发库，安装和卸载工具箱

□ 方式1：可视化操作

安装操作步骤：

1. 注册库中点击“添加包”
2. 搜索第三方工具箱，选择对应工具箱，添加至注册库列表
3. 在注册库列表中右击，选择“安装”

卸载操作步骤：

1. 注册库中右击包，选择卸载

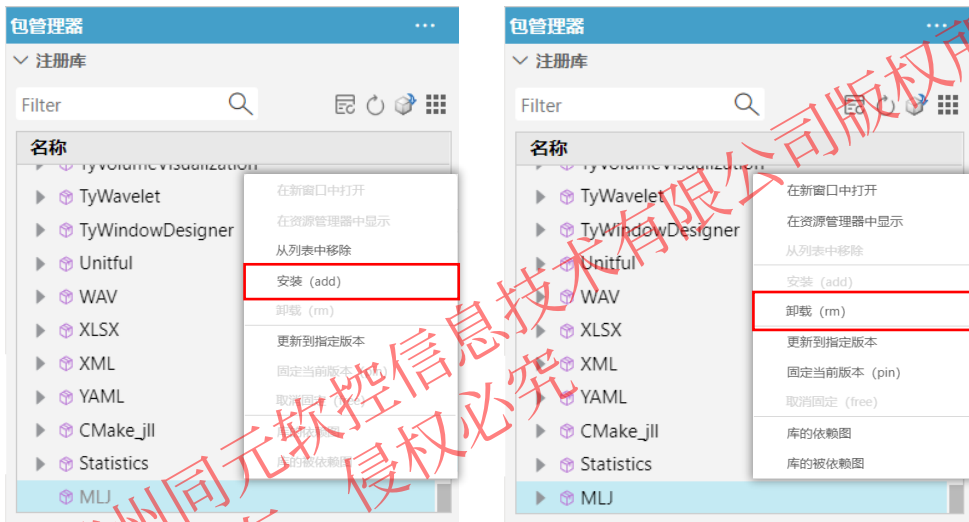
□ 方式2：文本操作

安装操作步骤：

1. 在REPL中输入 `1` 进入pkg模式(退出pkg模式可使用Backspace或Ctrl + C)
2. 在pkg模式中输入： `add 第三方工具箱名`

卸载操作步骤：

1. 在pkg模式中输入： `remove 第三方工具箱名`



```
(@v1.9) pkg> add MLJ
```

```
(@v1.9) pkg> remove MLJ
```

```
(@v1.9) pkg> add MLJ
```

安装

```
(@v1.9) pkg> remove MLJ
```

卸载

说明：

- 同元商业工具箱无需安装，可直接使用

2.3 字体显示

点击软件**设置-首选项**，索引到**Julia命令行窗口-终端显示**，更改**终端**字号大小



终端数据格式显示设置：首选项里提供3种设置，分别为自适应(default)、长固定(long)、短固定(short)，可根据需求进行选择



@苏州同元软控信息技术有限公司版权所有，侵权必究

PART 03 ➔

脚本构建

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

03

3.1 交互式编程环境

Syslab提供交互式编程环境，可在命令行窗口(REPL)中可直接输入命令，并得到结果。

操作步骤：

- 在julia>后输入命令，敲击“Enter”，Syslab自动执行并得到执行结果
- 定义变量a，并修改变量a输出显示格式

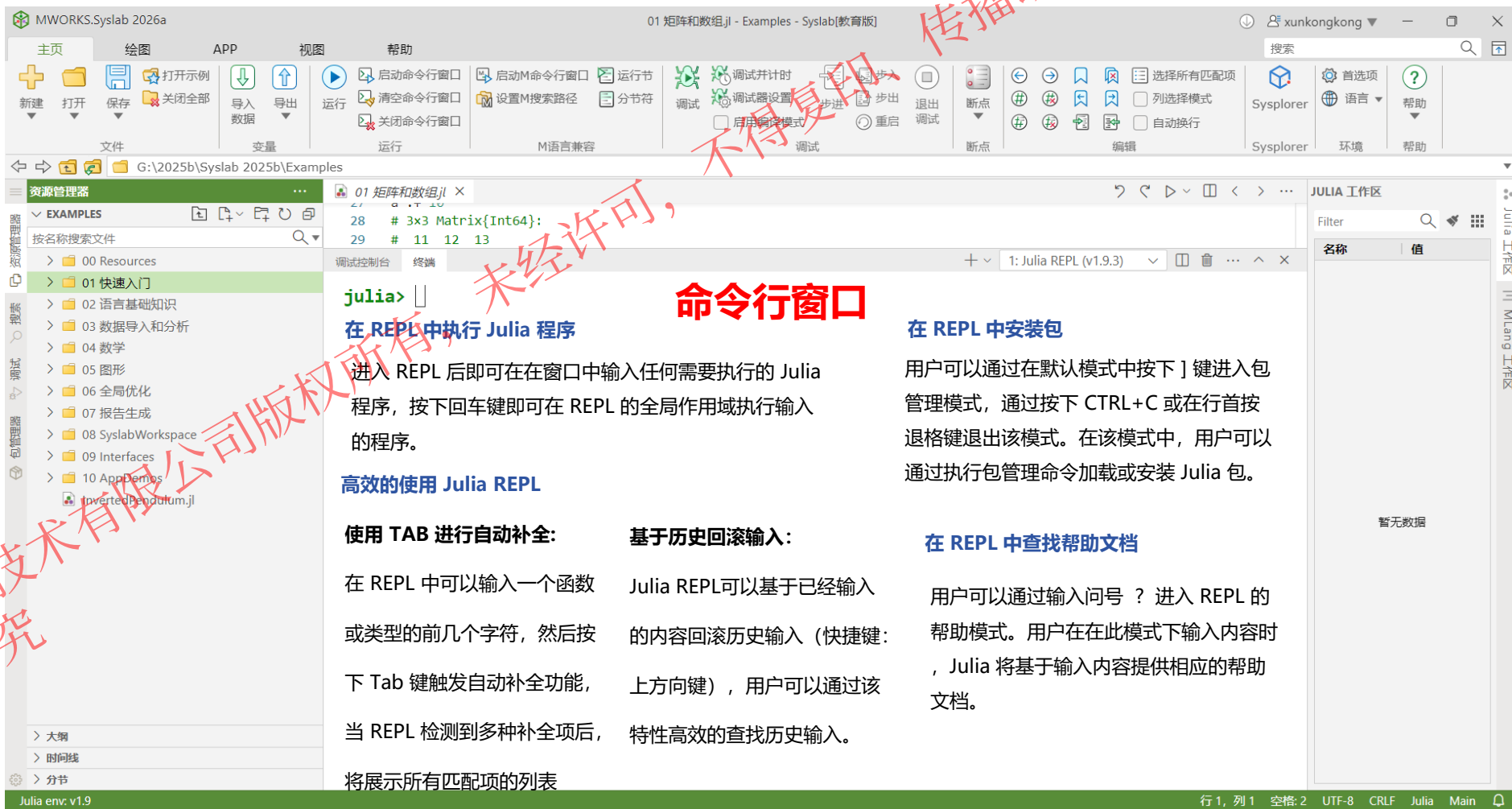
```
julia> a=sin(2)
0.9092974268256817
```

```
julia> ty_format("short",a)
0.9093
```

```
julia> ty_format("long",a)
0.909297426825682
```

Tips:

1. ty_format函数用来设置变量输出显示格式
2. 键盘“↑”“↓”可以找到历史指令
3. 利用键盘上的左右键，将光标移动到需要进行操作的字符段处，再进行复制粘贴，或是除操作



命令行窗口

在REPL中执行 Julia 程序

进入 REPL 后即可在窗口中输入任何需要执行的 Julia 程序，按下回车键即可在 REPL 的全局作用域执行输入的程序。

高效的使用 Julia REPL

使用 TAB 进行自动补全:

在 REPL 中可以输入一个函数或类型的前几个字符，然后按下 Tab 键触发自动补全功能，当 REPL 检测到多种补全项后，将展示所有匹配项的列表

基于历史回滚输入:

Julia REPL可以基于已经输入的内容回滚历史输入（快捷键：上方向键），用户可以通过该特性高效的查找历史输入。

在 REPL 中安装包

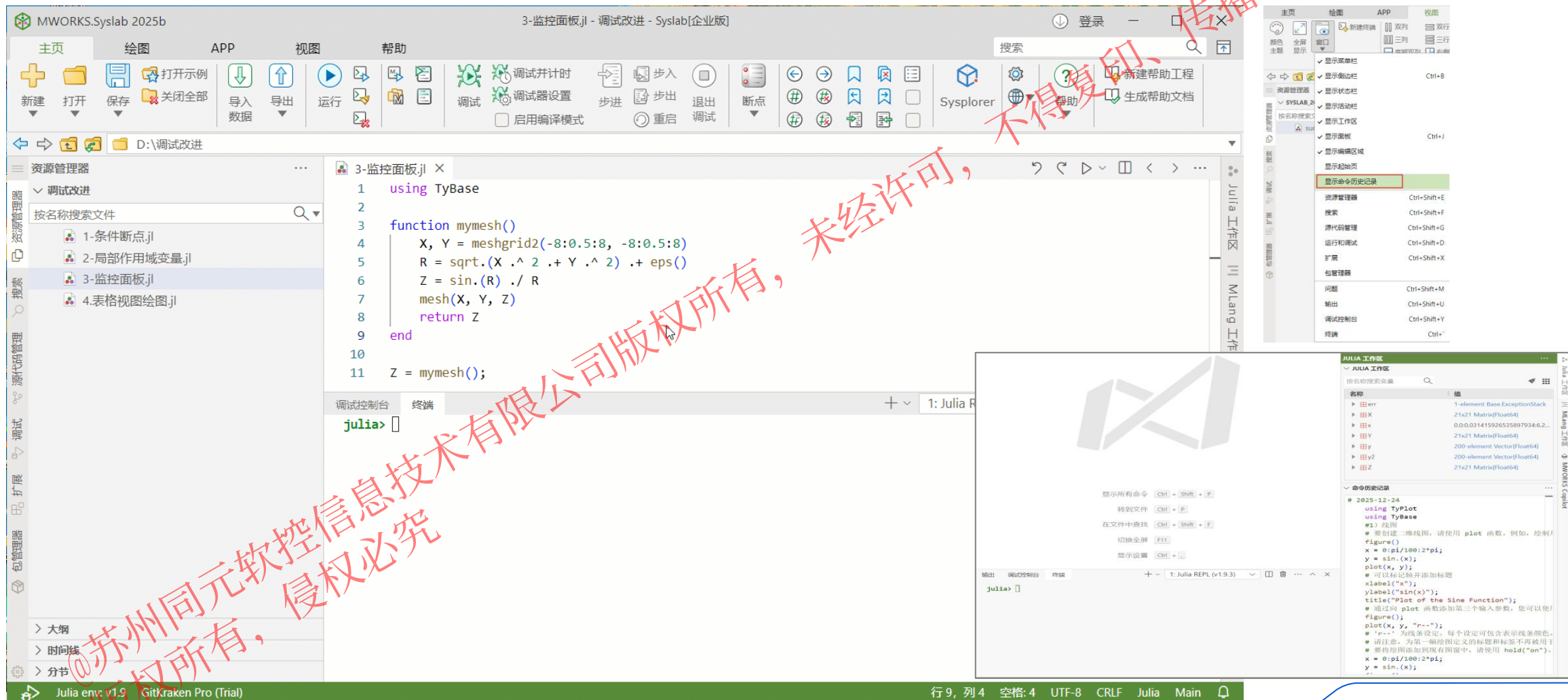
用户可以通过在默认模式中按下] 键进入包管理模式，通过按下 CTRL+C 或在行首按退格键退出该模式。在该模式中，用户可以通过执行包管理命令加载或安装 Julia 包。

在 REPL 中查找帮助文档

用户可以通过输入问号 ? 进入 REPL 的帮助模式。用户在此模式下输入内容时，Julia 将基于输入内容提供相应的帮助文档。

3.1 交互式编程环境

Julia 命令行窗口：新增命令历史记录面板(视图-窗口-显示命令历史记录)，命令行窗口支持鼠标点击插入光标、选中删除或替换等功能

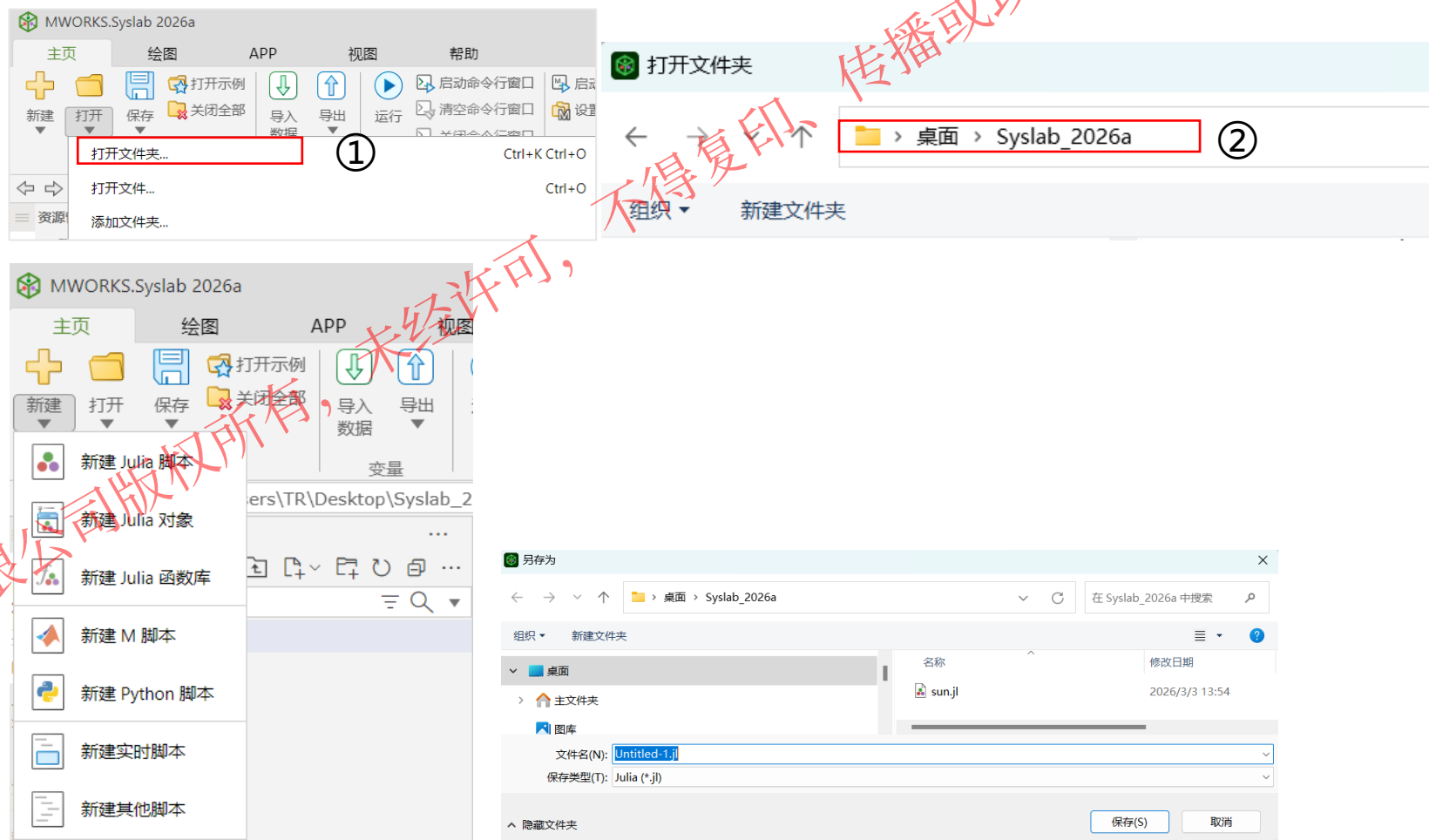


3.2 文件管理

打开相关文件夹作为工作目录，并在资源管理器中管理文件。资源管理器上方的路径导航栏功能，实时显示当前工作目录路径

操作步骤：

- 选择工作目录：主页-打开-“打开文件夹”
- 选择已有文件夹



Tips: 新建Python或M脚本，Syslab可以使用Python或M环境，Syslab内置Python环境版本为3.11.4

3.2 文件管理

打开相关文件夹作为工作目录，并在资源管理器中管理文件。资源管理器上方的路径导航栏功能，实时显示当前工作目录路径



新建文件...	
新建文件夹...	
在文件资源管理器中显示	Shift+Alt+R
在集成终端中打开	
在此窗口中打开	
在文件夹中查找...	Shift+Alt+F
剪切	Ctrl+X
复制	Ctrl+C
粘贴	Ctrl+V
复制路径	Shift+Alt+C
复制相对路径	Ctrl+K Ctrl+Shift+C
重命名...	F2
删除	Delete
关闭文件夹	
Julia: 激活全局环境	
Julia: 设为当前工作目录	

➤ 新建脚本文件-sun

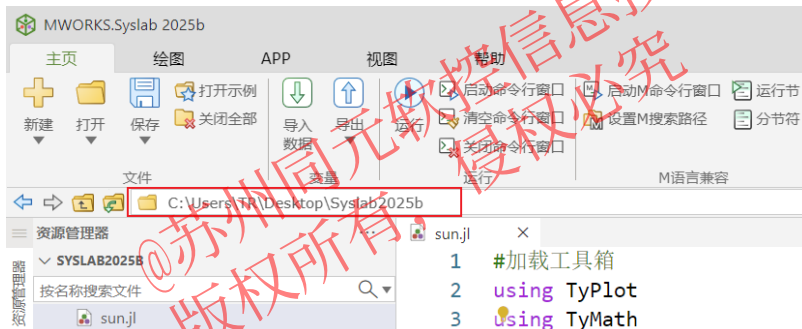


- 工作目录下新建脚本：M或者Julia
- 工作目录下新建文件夹

➤ 点击文件右键对文件进行重命名

➤ 不需要的脚本可以delete或 右击选择删除

➤ 返回上一级：可以返回上级文件夹



路径导航栏功能，实时显示当前工作目录路径

3.3 代码编辑

#加载工具箱

using TyPlot

using TyMath

#定义变量

dec = 23.4 # 太阳偏角: 23° 24'

lat = 31 + 30 / 60 # 纬度: 31° 30'

#使用函数转换弧度

dec = deg2rad(dec) # 将角度转换为弧度

lat = deg2rad(lat)

#定义时间变变量

默认时间从早上5点开始, 到19点结束

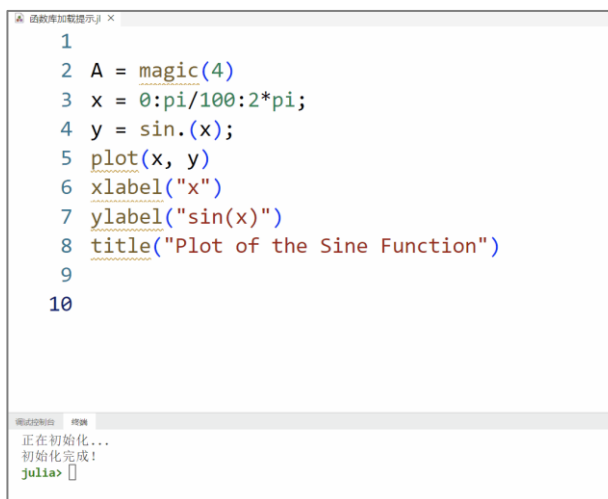
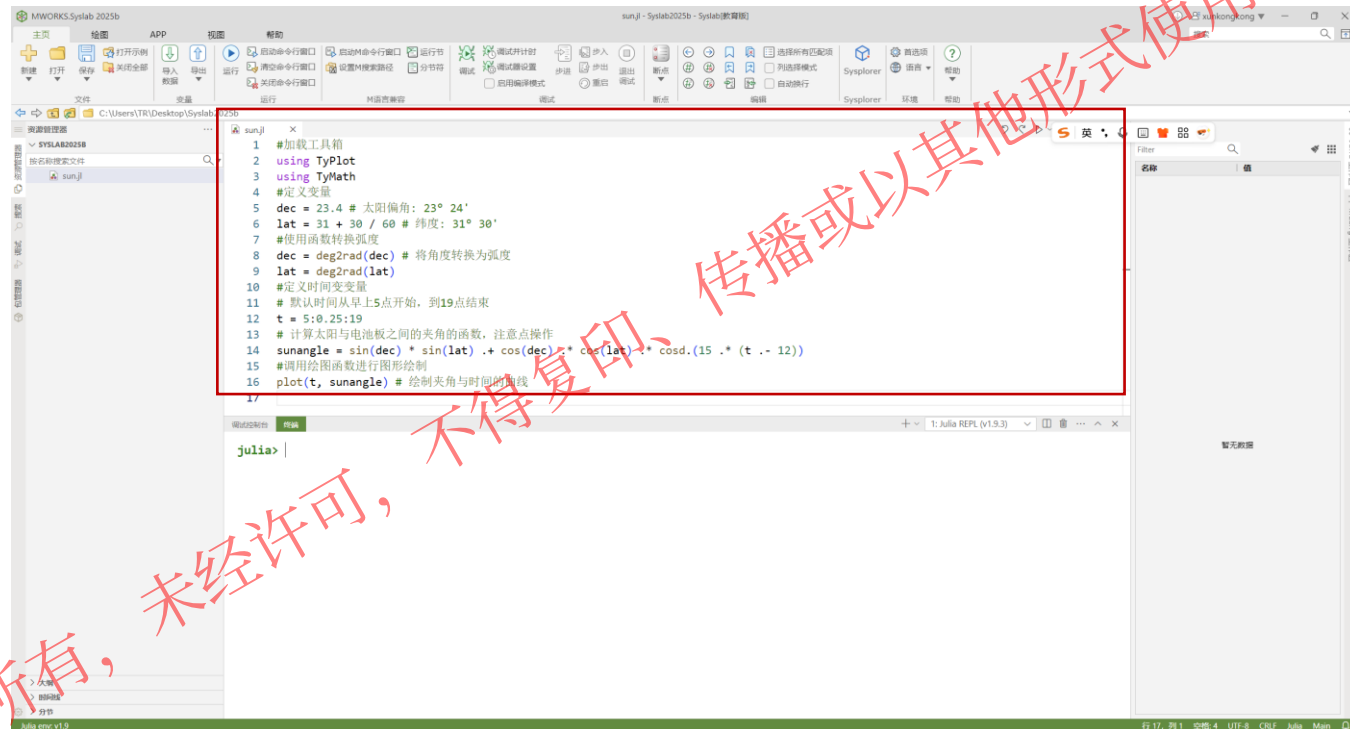
t = 5:0.25:19

计算太阳与电池板之间的夹角的函数, 注意点操作

sunangle = sin(dec) * sin(lat) .* cos(dec) .*
cos(lat) .* cosd.(15 .* (t .- 12))

#调用绘图函数进行图形绘制

plot(t, sunangle) # 绘制夹角与时间的曲线



函数库加载提示:

- 对于从资源管理器中打开的Julia脚本, 会检查该文件中未定义的库函数, 以波浪线提示, 并提供快速修复功能

3.3 代码编辑-函数构建及调用

示例：计算A、B、C多个矩阵的乘积

$$A = \begin{vmatrix} 1 & 2 \\ 3 & 4 \end{vmatrix} \quad B = \begin{vmatrix} 5 & 6 \\ 7 & 8 \end{vmatrix} \quad C = \begin{vmatrix} 9 \\ 10 \end{vmatrix}$$

计算多个矩阵的乘积

```
function multiply_matrices_test(matrices::Vector{Array{Float64}})
    result = matrices[1]
    for i in 2:length(matrices)
        result = result * matrices[i]
    end
    return result
end
```

定义矩阵A、B、C

A = [1.0 2.0; 3.0 4.0]

B = [5.0 6.0; 7.0 8.0]

C = [9.0; 10.0]

matrices = [A, B, C]

result = multiply_matrices_test(matrices)

主函数与子函数在同一脚本

输出结果为：

2-element Vector{Float64}:
391.0
887.0

3.3 代码编辑-函数构建及调用

示例：计算A、B、C多个矩阵的乘积

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \quad C = \begin{bmatrix} 9 \\ 10 \end{bmatrix}$$

计算多个矩阵的乘积

```
function multiply_matrices_test(matrices::Vector{Array{Float64}})
    result = matrices[1]
    for i in 2:length(matrices)
        result = result * matrices[i]
    end
    return result
end
```

Julia 脚本编辑好 function 之后另存为时支持自动识别函数名保存

```
include("multiply_matrices.jl")
```

实现函数脚本调用

```
A = [1.0 2.0; 3.0 4.0]
```

```
B = [5.0 6.0; 7.0 8.0]
```

```
C = [9.0; 10.0]
```

```
matrices = [A, B, C]
```

```
result = multiply_matrices_test(matrices)
```

主函数与子函数在不同脚本



形成子函数文件



构建主函数文件

3.3 代码编辑-函数构建及调用

`include("函数文件路径")` 实现函数脚本调用

□ 资源管理器打开至函数文件所在目录



- 路径 ✓

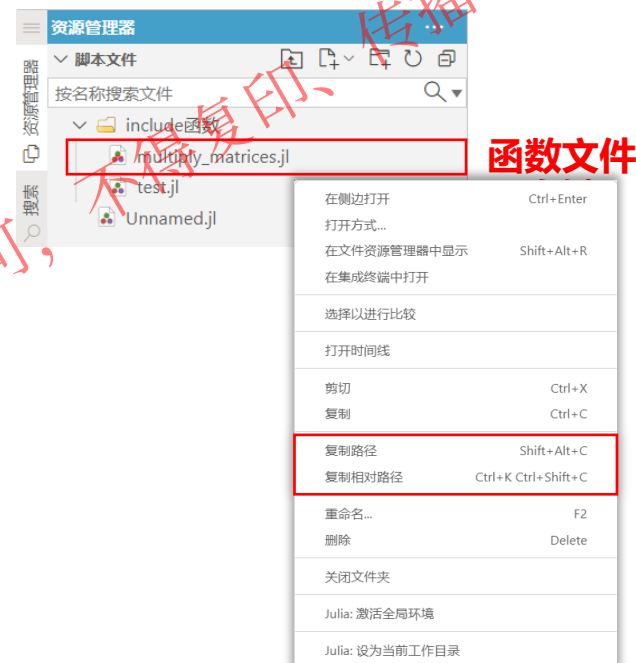
C:/Users/TR/Desktop/.../脚本文件/include函数/multiply_matrices.jl

- 相对路径 ✓

multiply_matrices.jl

提示: `pwd()` 命令也可显示当前文件夹的路径, 通过拼接构成绝对路径 `include(pwd()*"/multiply_matrices.jl")`

□ 资源管理器未打开至函数文件所在目录



- 路径 ✓

C:/Users/TR/Desktop/.../脚本文件/include函数/multiply_matrices.jl

- 相对路径 ✗

include函数/multiply_matrices.jl

3.3 代码编辑-函数构建及调用

注意事项

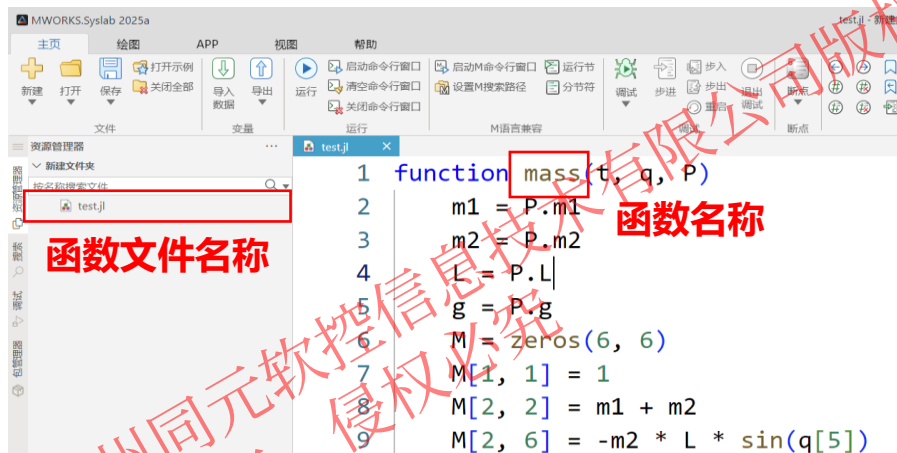
□ 关于函数文件名称

M软件:

- 函数文件名称必须与文件中定义的主函数名称完全一致
例如: 若函数定义为 `function y = CalSum(x)`, 则文件必须保存为 `CalSum.m`, 否则会报错

Syslab:

- 函数文件名称与文件中定义的函数名称无关联



Syslab

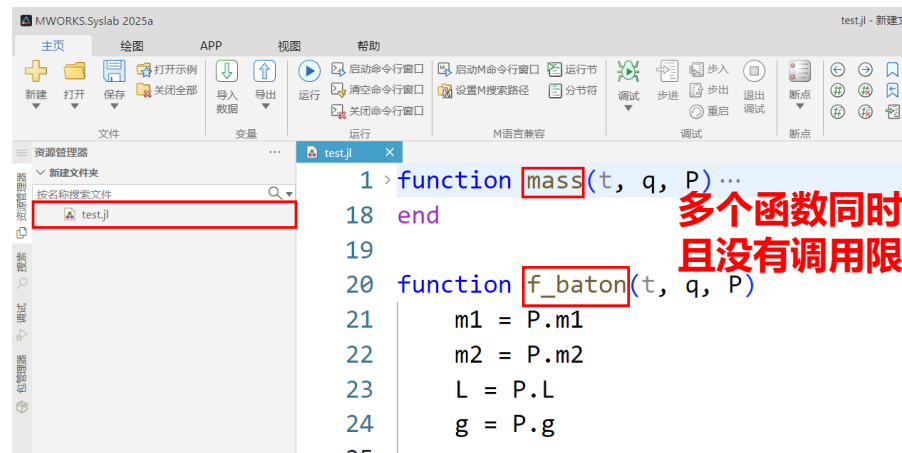
□ 关于函数文件中定义多个函数

M软件:

- 函数文件支持一个主函数+多个子函数的结构
- 每个函数文件只能有一个主函数, 其名称必须与文件名一致
- 在主函数之后, 可以定义多个子函数, 子函数仅在该文件内可见, 无法被外部直接调用

Syslab:

- 函数文件可定义多个函数, 没有主函数或子函数的严格限制



Syslab

3.3 代码编辑-控制流与循环

for语句为例

一般格式(单个迭代变量):

格式1:

```
for 迭代变量=可迭代数集
#语句块
end
```

格式2:

```
for 迭代变量 in 可迭代数集
#语句块
end
```

格式3:

```
for 迭代变量 ∈ 可迭代数集
#语句块
end
```

```
julia> for i = 1:5
        print(i)
      end
12345

julia> for i in [1 2 3 4 5]
        print(i)
      end
12345

julia> for i ∈ (1, 2, 3, 4, 5)
        print(i)
      end
12345
```

```
s = 0
for i = 1:10
    s = i
    println(s) # 若修改 s, 则在非交互和交互的上下文中,
               # 表现不一致, 会出现歧义
end
```

```
julia> 正在运行 for.jl
Warning: Assignment for 's' in soft scope is ambiguous because a global variable by the same name exists: 's' will be treated as
a new local. Disambiguate by using 'local s' to suppress this warning or 'global s' to assign to the existing global variable.
@ . C:\Users\W\Desktop\Syslab2025b\for.jl:4
1
2
3
4
5
6
7
8
9
10
```

for 作用域全局变量遮蔽告警功能

局部变量 's' 隐藏了一个同名的全局变量

for.jl(1, 1): 隐藏的全局变量见此处

```
4 | println(s) # 若修改 s, 则在非交互和交互的上下文中, 表现不一致, 会出现歧义
5 | end
```

```
s = 0
for i = 1:10
    global s
    s = i
    println(s) # 若修改 s, 则在非交互和交互的上下文中,
               # 表现不一致, 会出现歧义
end
```

Syslab使用手册

- 总纲: 每个模块会创建一个新作用域, 与所有其他模块的全局作用域分开。
- 局部作用域: 大多数代码块都引入了新的作用域。如果 x 还不是局部变量, 则在全局作用域中查找一个名为 x 的局部变量。
- 软作用域: 在 x = module() 函数调用中引入。如果 x 还不是局部变量, 行内函数会查找一个名为 x 的局部变量。
- 硬作用域: 在 x = module() 函数调用中引入。如果 x 还不是局部变量, 行内函数会查找一个名为 x 的局部变量。
- 如果全局变量 x 被定义, 此次函数调用会创建一个新的名为 x 的局部变量。
- 如果全局变量 x 被定义, 此次函数调用会创建一个新的名为 x 的局部变量。
- 在非交互上下文中 (文件, eval) 中, 会创建一个新的作用域。
- 在交互上下文中 (REPL, notebooks) 中, 会创建一个新的作用域。

引入作用域的结构

结构	作用域类型	允许使用
module, baremodule	全局	全局
struct	局部 (软)	全局
for, while, try	局部 (软)	global, local
macro	局部 (硬)	全局
函数, do 语句块, let 语句块, 函数体, 生成器	局部 (硬)	global, local

局部作用域

for 循环作用域

```
# 示例 1: 在局部作用域 "for" 中定义变量
x = 100
for i = 1:10
    x = i
    println(x)
end

# 示例 2: 在局部作用域 "for" 中定义变量
x = 100
```

@苏州同元软控信息技术有限公司版权所有, 侵权必究

3.4 代码运行

运行当前文件

- 主页工具栏运行按钮
- 快捷键：Ctrl+F5

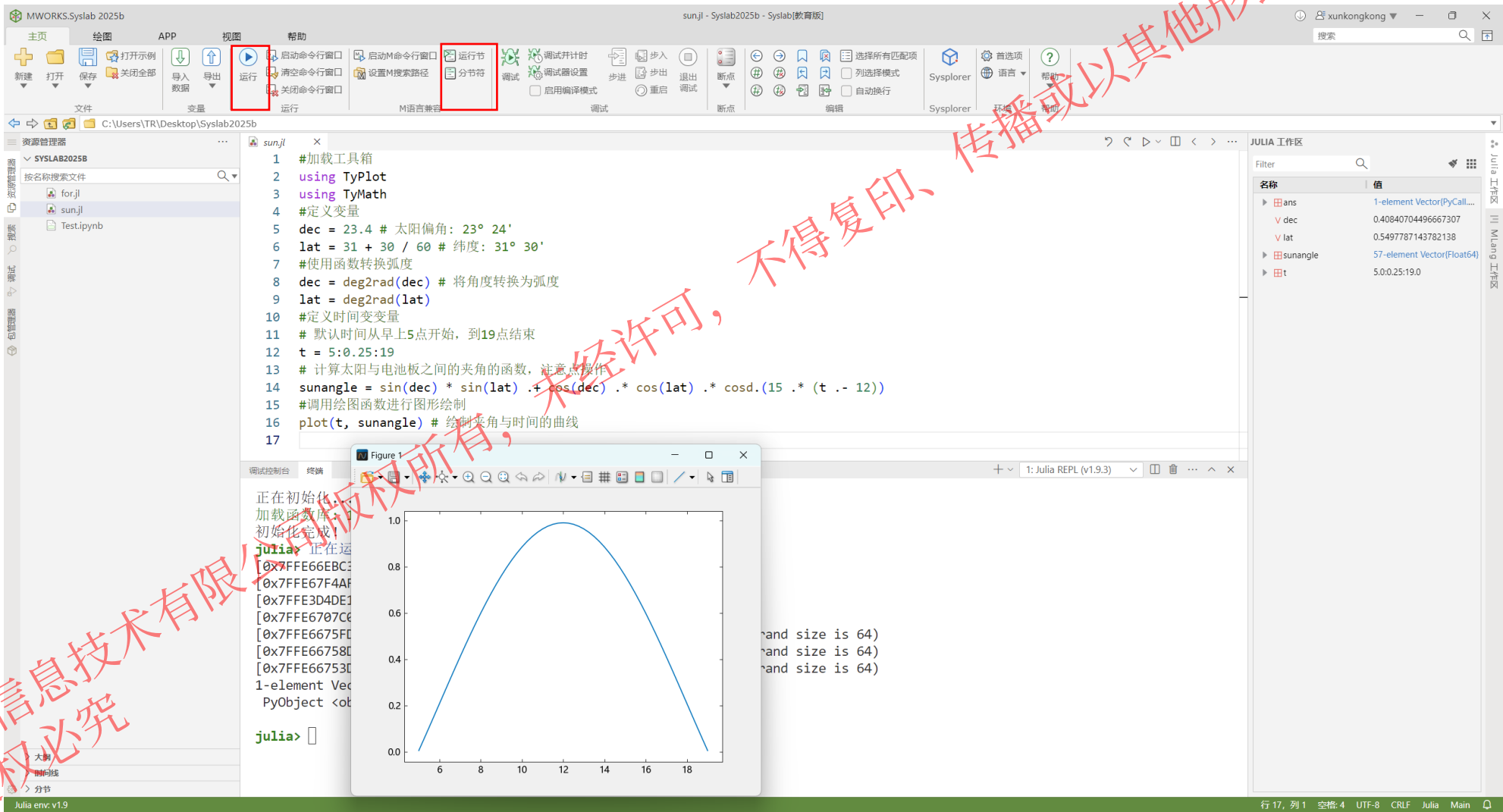
运行部分代码

方法一

- ①：左键选中需要运行的代码段落，例如阴影部分
- ②：使用：Ctrl+enter

方法二

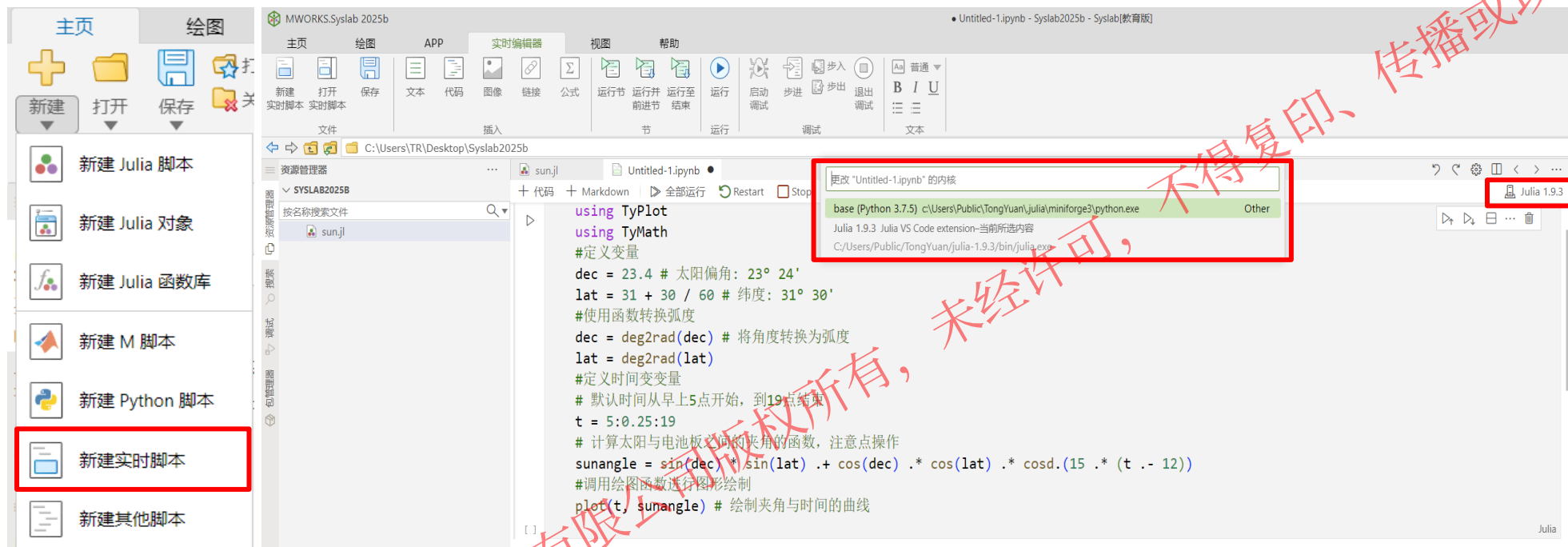
- ①：代码前用##标识/分节符
- ②：运行当前节



示例见：本课件第6章节示例

3.5 实时脚本

支持用户在统一的文档环境中将代码与嵌入式输出、格式化文本、方程和图像组合到一起，生成可交互式记事本，并与他人分享



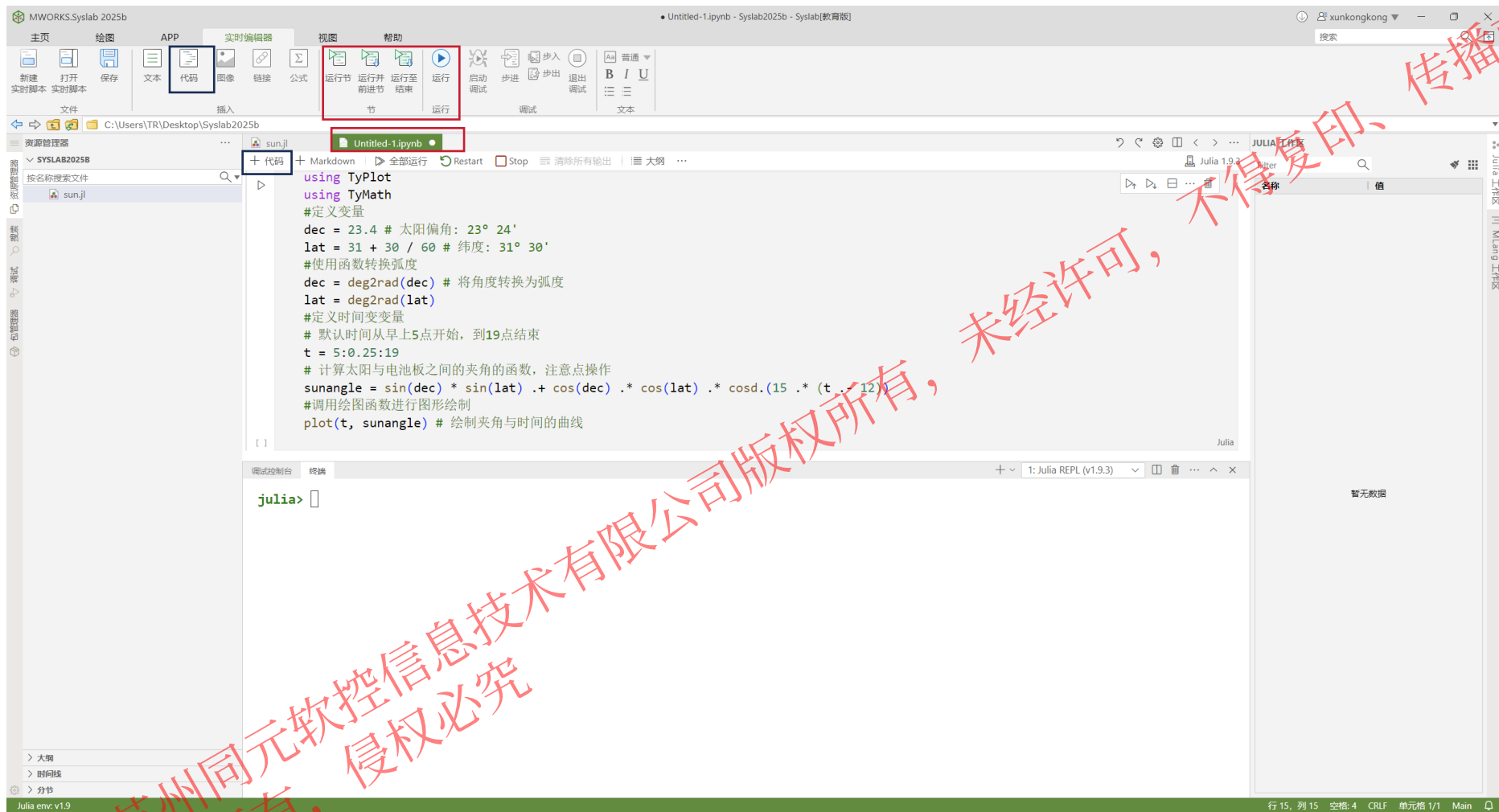
Tips: 此处编写的Julia脚本默认不加载任何工具箱，需要自己using XXX函数库，包括同元函数库

注意:

新建或打开实时脚本，启动实时编辑器功能

3.5 实时脚本

支持编辑并运行 **Julia**、**Python** 脚本语言



新建Julia / Python代码:

• 方式1:



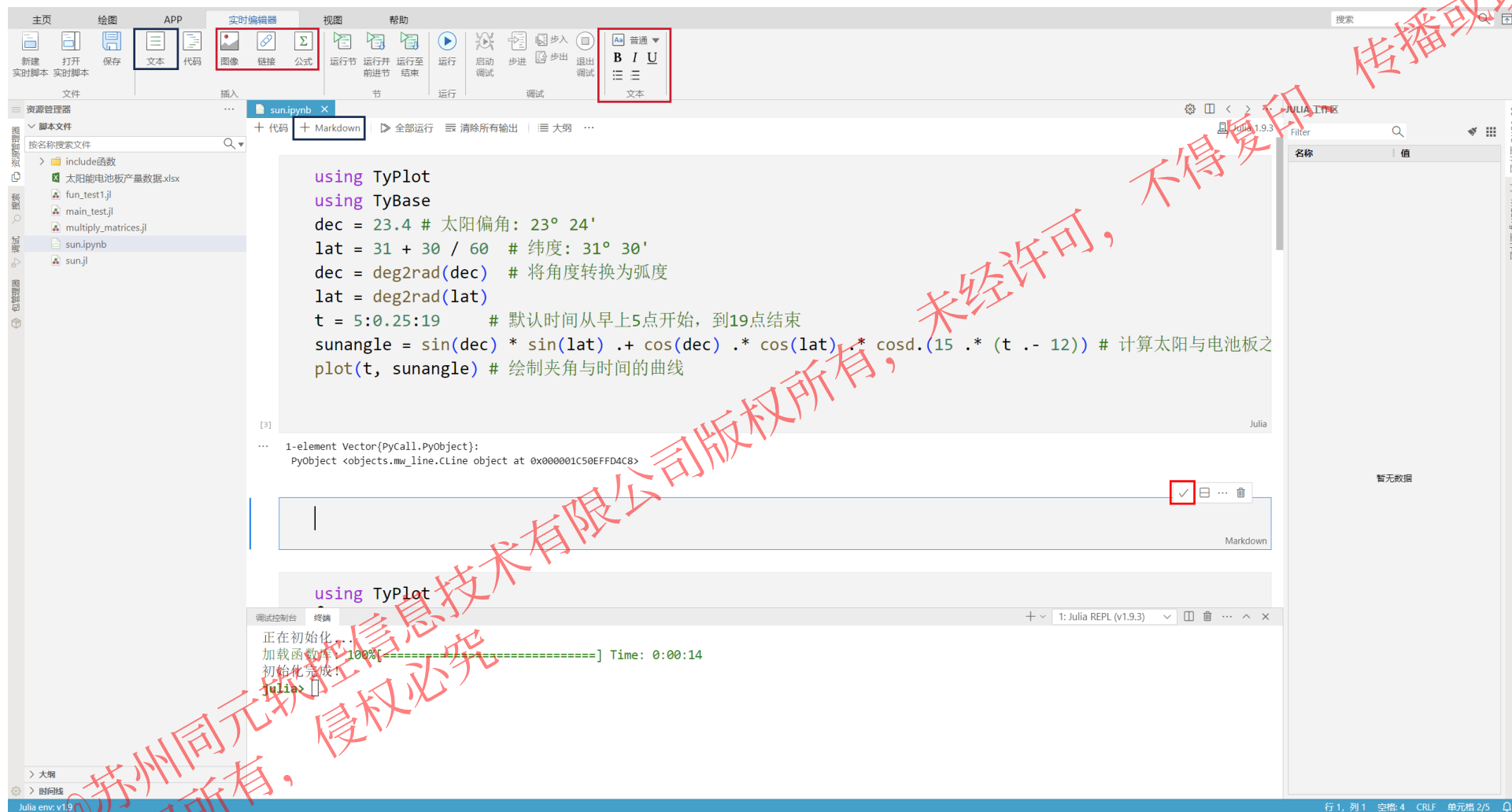
• 方式2: + 代码

代码运行:

- **运行节**: 运行当前节
- **运行并前进节**: 运行当前节并前进到下一节
- **运行至结束**: 从当前节运行至代码结束
- **运行或全部运行**: 运行全部代码
- **停止**: 终止运行进程

3.5 实时脚本

可新建并编辑Markdown文本，并进行预览



新建Markdown文本:

- 方式1:  文本
- 方式2: + Markdown

说明:

- 文本中遵循Markdown语法
- 可直接通过可视化操作编辑Markdown
- ☒ 进入Markdown预览, 入Markdown编辑

PART 04 ➔

代码调试

@苏州同元软控信息技术有限公司版权所有，侵权必究

04
不得复印、传播或以其他方式使用

5.1 代码调试

代码编写很难一次成功，往往需要进行代码调试，列举2个常见的错误情况

□ 情况1-代码运行报错

```
test.jl × multiply_matrices.jl
1 include("multiply_matrices.jl")
2 # 定义矩阵A、B、C
3 A = [1.0 2.0; 3.0 4.0]
4 B = [5.0 6.0; 7.0 8.0]
5 C = [9.0; 10.0] 逗号改为分号
6 matrices = [A; B; C]
7 result = multiply_matrices_test(matrices)
```

运行报错

调试控制台 终端

报错原因：列数不匹配

ERROR: ArgumentError: number of columns of each array must match (got (2, 2, 1))

Stacktrace:

```
julia> 正在运行 test.jl
[1] _typed_vcat{#unused#::Type{Float64}, A::Tuple{Matrix{Float64}, Matrix{Float64}, Vector{Float64}}}
 @ Base .\abstractarray.jl:1689
[2] typed_vcat
 @ .\abstractarray.jl:1703 [inlined]
[3] vcat{::Matrix{Float64}, ::Matrix{Float64}, ::Vector{Float64}}
 @ Base .\array.jl:1958
[4] top-level scope
 @ c:\Users\TR\Desktop\include函数\test.jl:6
```

出错代码位置：第6行

□ 情况2-代码运行不报错，结果错误

```
test.jl × multiply_matrices.jl ×
1 # 计算多个矩阵的乘积
2 function multiply_matrices_test(matrices::Vector{Matrix{Float64}})
3     result = matrices[1]
4     for i in 1:length(matrices)
5         result = result * matrices[i]
6     end
7     return result
8 end
```

结果错误

调试控制台 终端

julia> 正在运行 test.jl

2-element Vector{Float64}:

```
2165.0
4721.0
```

正确结果为：

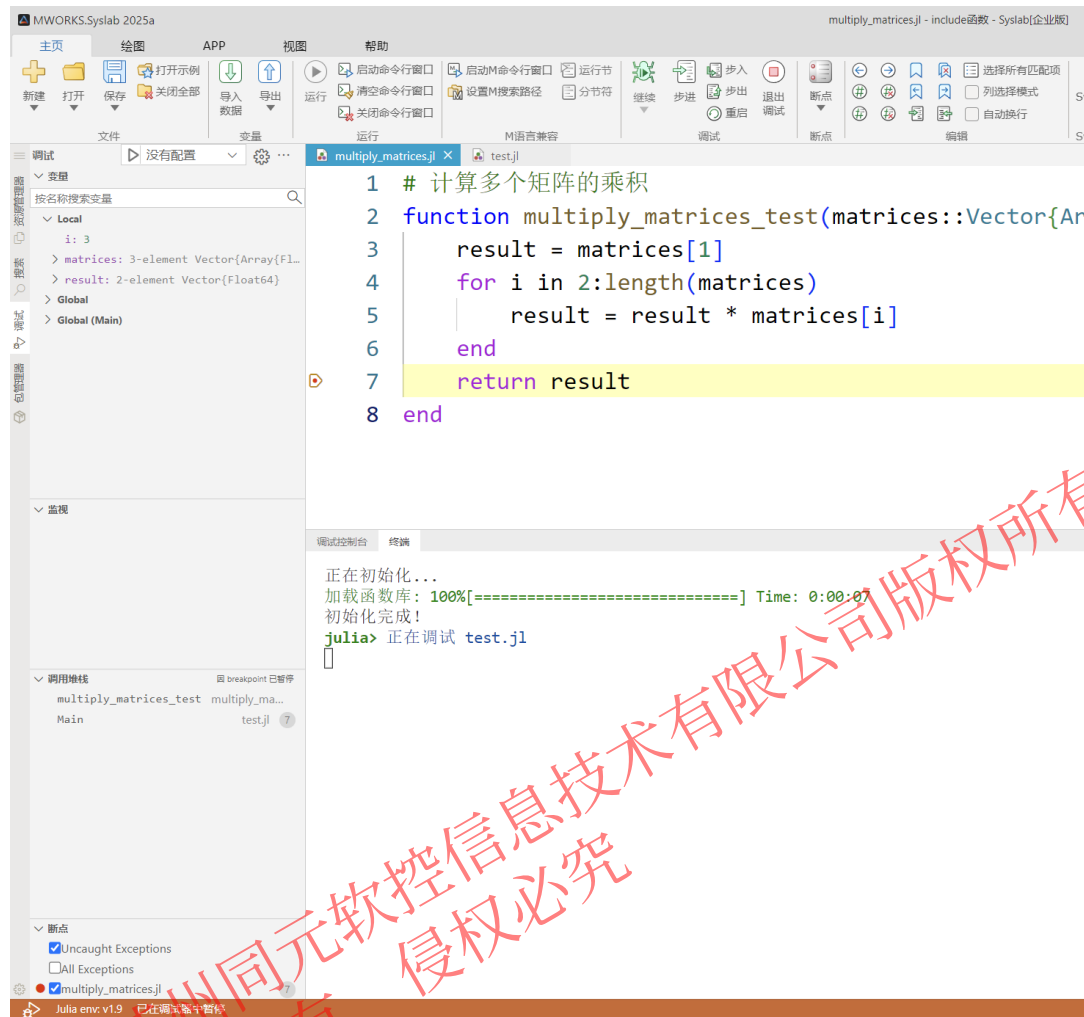
2-element Vector{Float64}:

```
391.0
887.0
```

说明：在此例中，情况1无需调试，情况2需调试

5.1 代码调试

Syslab提供代码调试器，支持对代码文件的**单步调试**、**断点调试**、**添加监视**、**查看调用堆栈**等，并提供**交互式调试控制台**



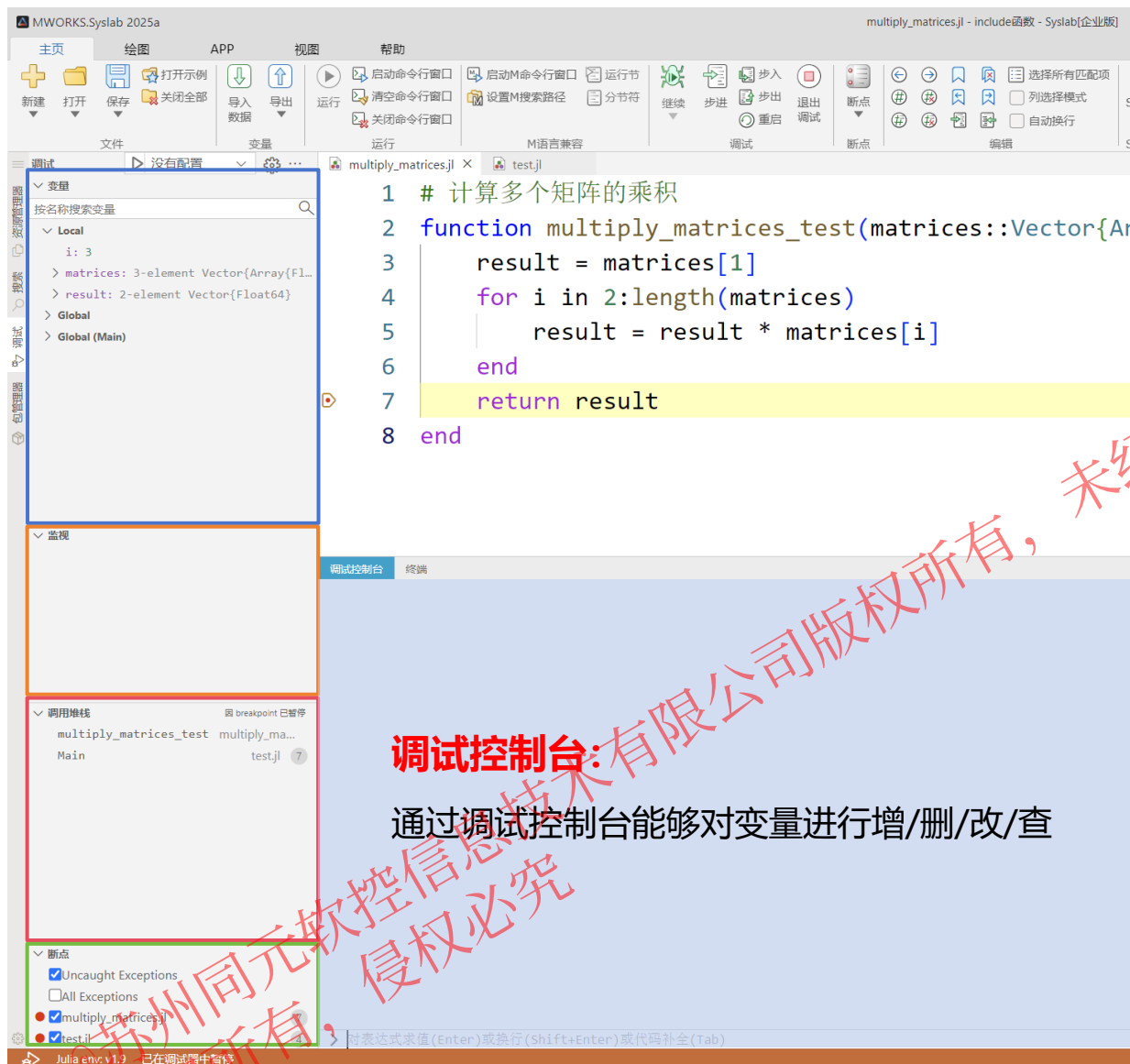
操作步骤:

1. 在脚本打上断点
2. 点击菜单栏中"启动调试"或F5进入DeBug模式

说明:

- **继续(F5)**: 继续运行调试
- **步进(F10)**: 单步执行遇到子函数时不会进入子函数内，而是将子函数整个执行完再停止
- **步入(F11)**: 单步执行遇到子函数就进入并且继续单步执行
- **步出(Shift+F11)**: 当单步执行到子函数内时，执行完子函数余下部分，并返回到上一层函数
- **重启(Ctrl+Shift+F5)**: 重新启动调试
- **退出调试(Shift+F5)**: 停止调试

5.1 代码调试



说明:

□ 变量:

- Local: 函数局部变量的值
- Global: 函数全局变量的值, 变量前具有global前缀
- Global(Main): 主程序全局变量的值

□ 监视: 输入变量名, 监视变量的变化

□ 调用堆栈: 用于了解函数间的调用关系和逻辑

□ 断点: 模型中所有断点的信息

调试控制台:

通过调试控制台能够对变量进行增/删/改/查

5.1 代码调试

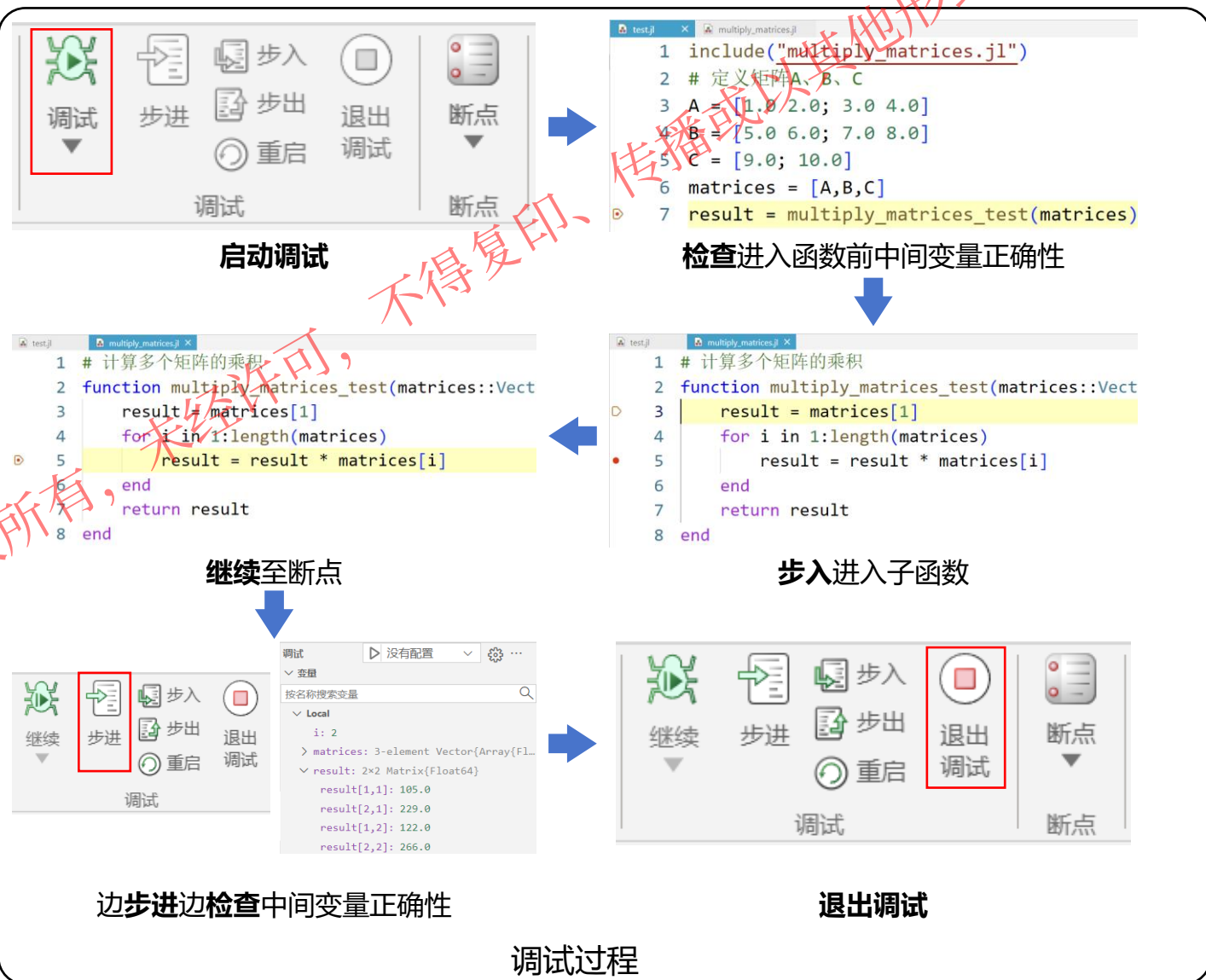
操作步骤:

1. 在脚本打上断点
2. 点击菜单栏中"启动调试"或F5进入DeBug模式

```
test.jl × multiply_matrices.jl
1 include("multiply_matrices.jl")
2 # 定义矩阵A、B、C
3 A = [1.0 2.0; 3.0 4.0]
4 B = [5.0 6.0; 7.0 8.0]
5 C = [9.0; 10.0]
断点1 matrices = [A,B,C]
• 7 result = multiply_matrices_test(matrices)
```

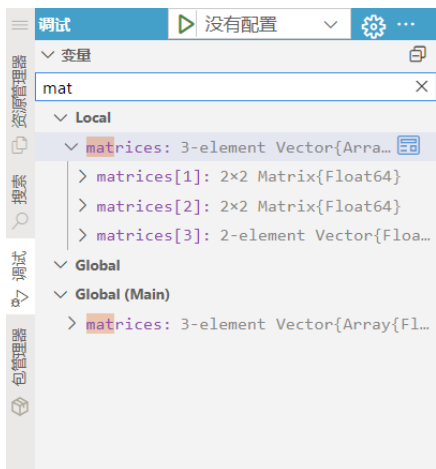
```
test.jl × multiply_matrices.jl ×
1 # 计算多个矩阵的乘积
2 function multiply_matrices_test(matrices::
3     result = matrices[1]
断点2 for i in 1:length(matrices)
5     result = result * matrices[i]
6 end
7 return result
8 end
```

打断点

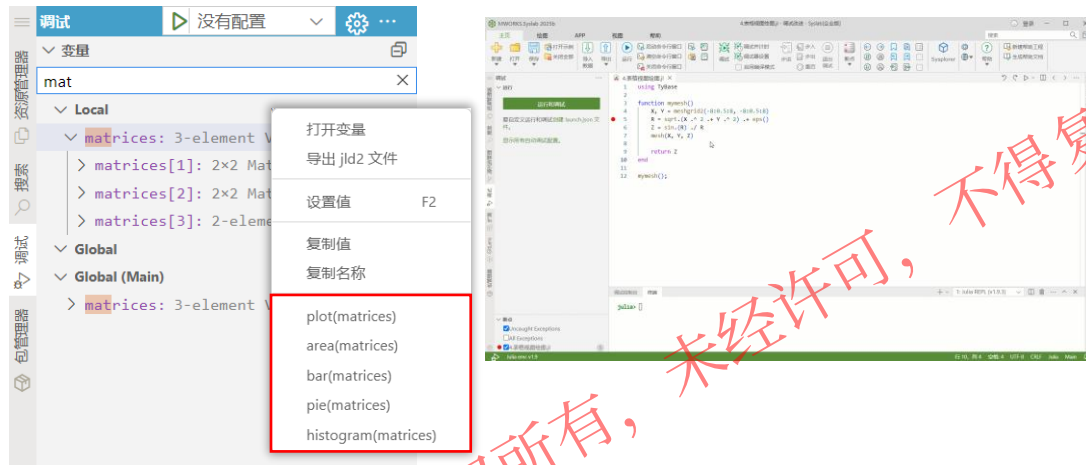


5.1 代码调试

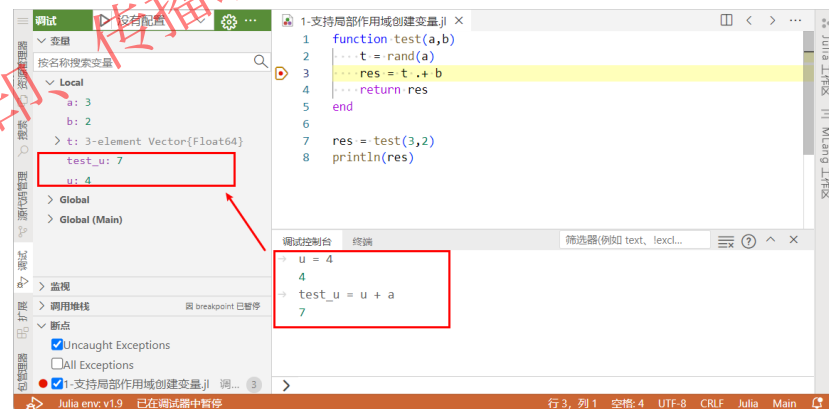
支持变量搜索



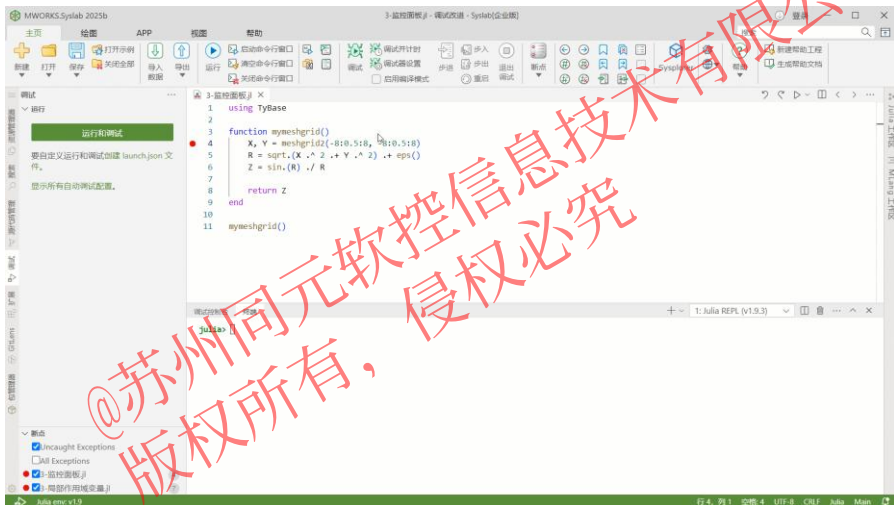
支持变量右键直接绘图和表格视图部分数据绘图



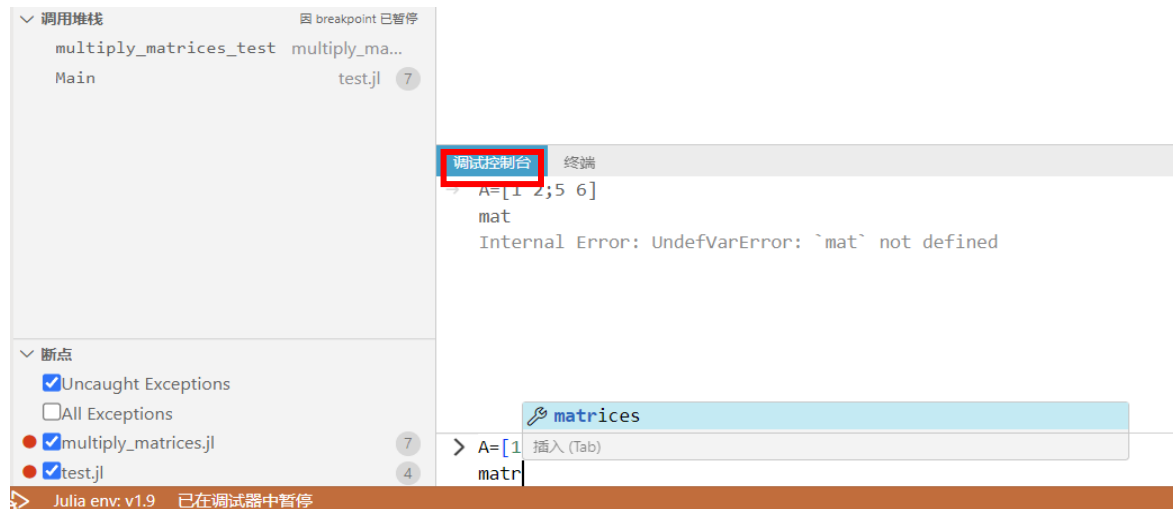
调试时支持在local局部作用域新增变量



支持监视面板打开变量编辑器表格视图，方便查看任意表达式值。



调试控制台支持换行输入、代码补全

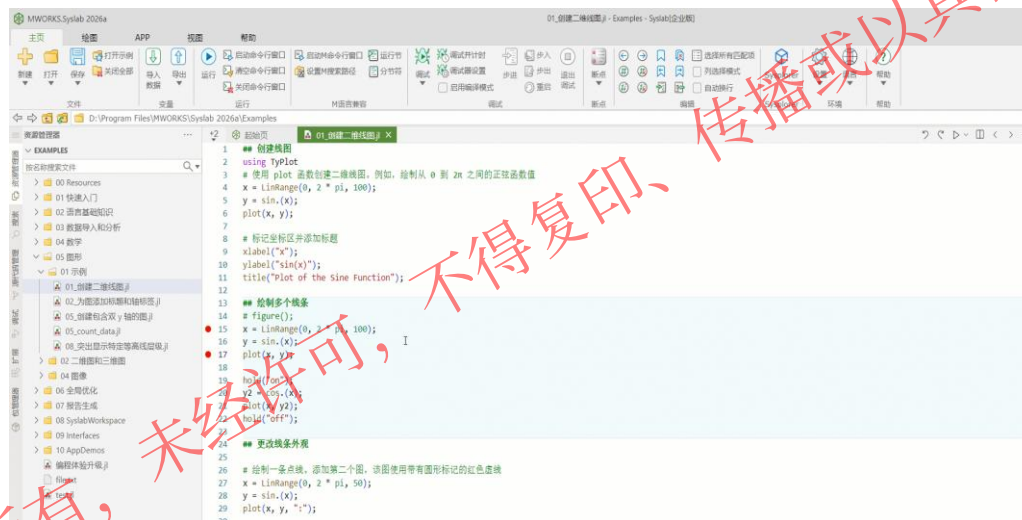
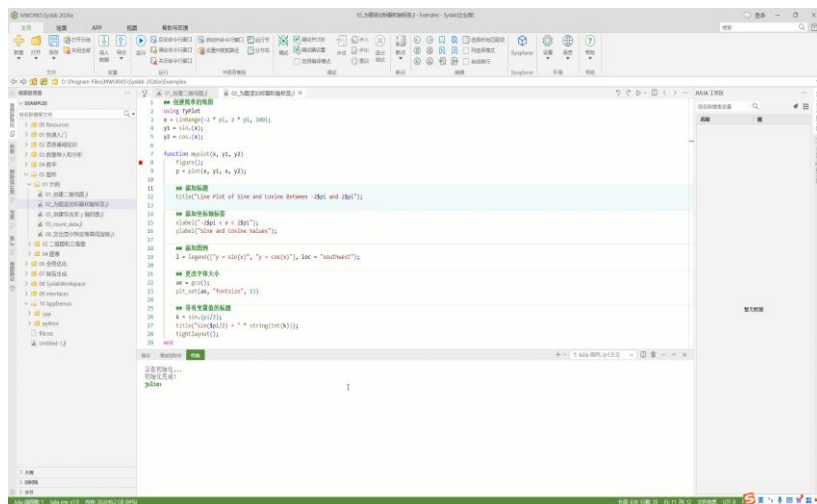


@苏州同元软控信息技术有限公司
版权所有，侵权必究

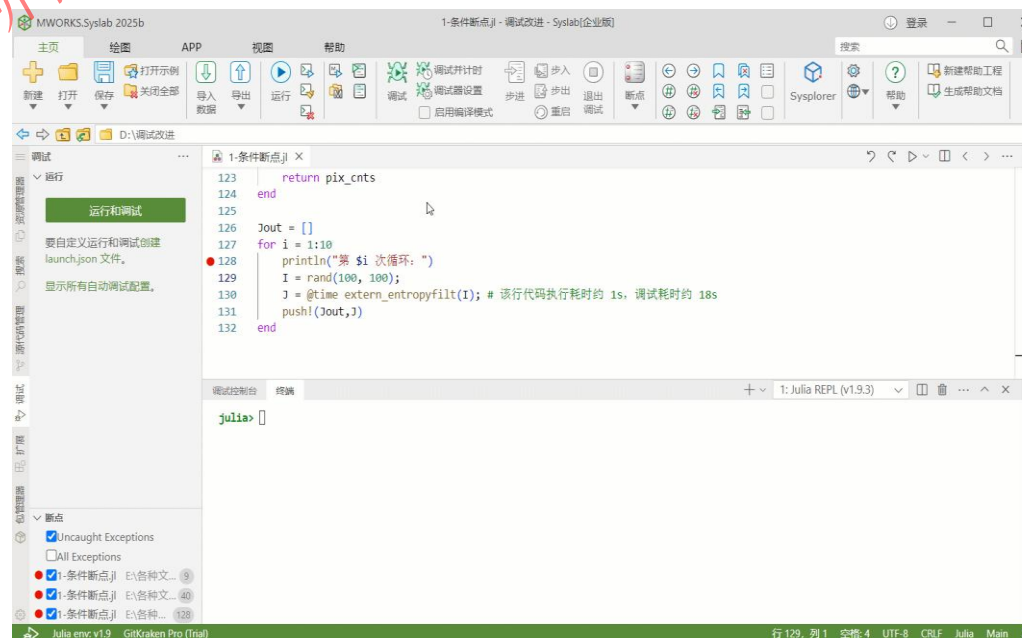
5.1 代码调试

□ 支持使用 @debug_func 调试指定函数

□ 支持分节调试



□ 新增 启用编译模式 选项，开启后从当前堆栈帧调用的函数以编译模式运行，能够大幅提升调试速度



@苏州同元软控信息技术有限公司
版权所有，侵权必究

5.2 性能分析

支持程序计时及程序性能采样分析

□ 提供多种宏调用方式，支持对表达式计时和查看内存分配情况

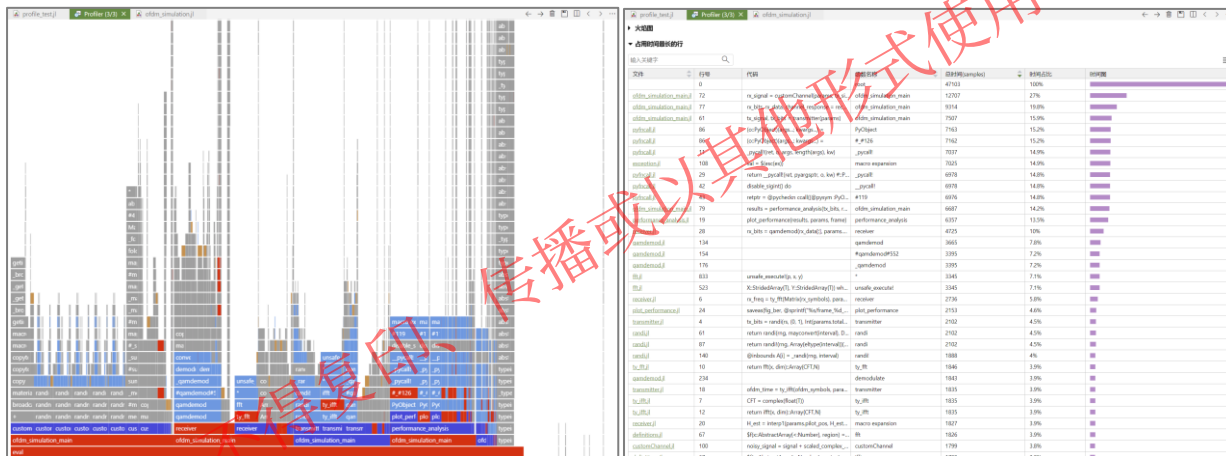
- **@time**对表达式计时和查看内存分配：运行时间；内存分配次数；总计分配的内存数；整理内存所用的时间
- **@elapsed**对表达式计时，以浮点数的形式返回执行所用的秒数。
- **@btime**提供在基准测试期间测量的最小运行时间。
- **@benchmark**用于对Julia表达式进行基准测试。其运行结果主要包括：运行次数、区分用户函数、同元函数、标准库函数、最小用时、最大用时、中位数用时、平均用时，以及占用内存等
- **@profview**对代码进行采样分析，并输出火焰图

```
time_test.jl
1 x = rand(10,10);
2
3 @time x * x; # 第一次存在编译时间
4
5 @time begin
6     x * x
7 end;

julia> 正在运行 time_test.jl
0.003447 seconds (57 allocations: 3.625 KiB, 96.75% compilation time)
0.000011 seconds (1 allocation: 896 bytes)
```

@time对表达式计时和查看内存分配

性能采样分析结果



支持“占用时间最长的行”表格，用于快速定位性能热点。

帮助文档-Julia高性能编程经验

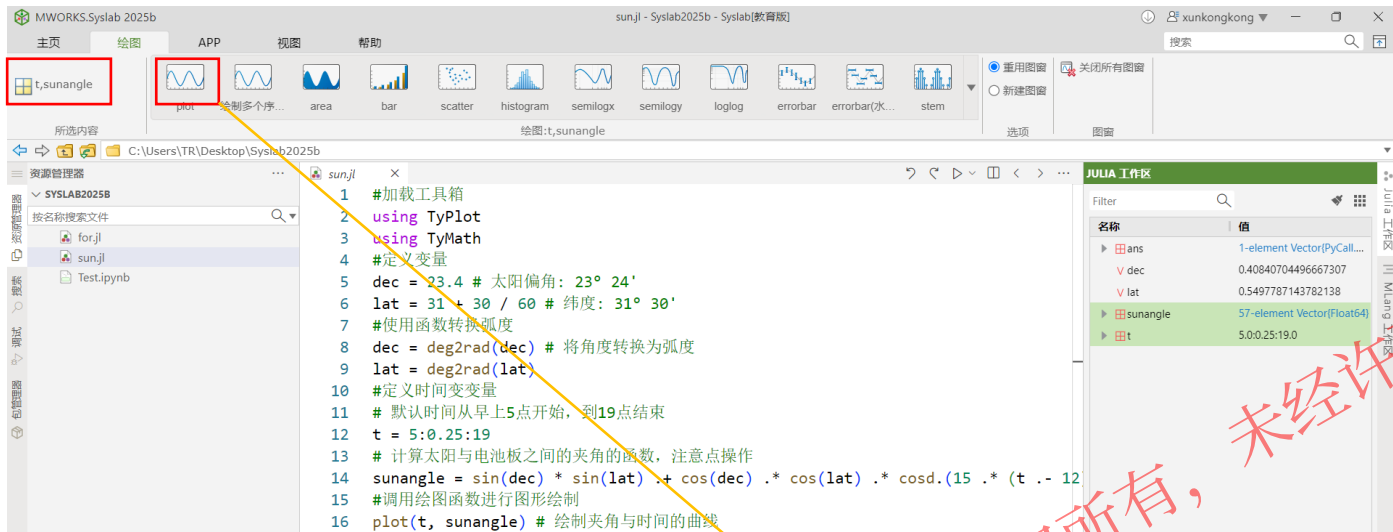
PART 05 ➔

结果查看

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
05

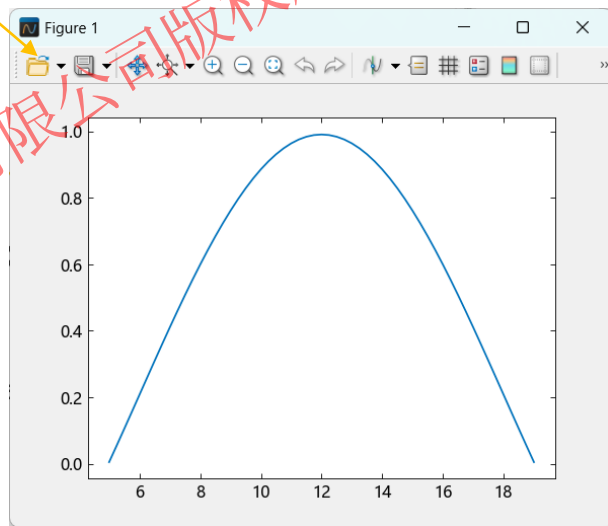
5.1 结果后处理-数据可视化(图形化操作)

绘图页中具备多种类型图形绘制功能，可直接选择变量进行绘制



选择工作区变量，可视化绘图步骤：

1. 工作空间左键选择绘图的变量
2. 在“绘图”菜单栏选择图类型

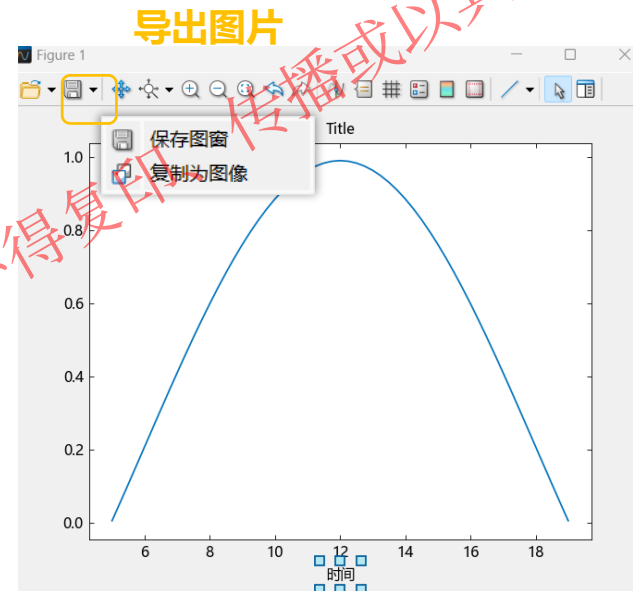
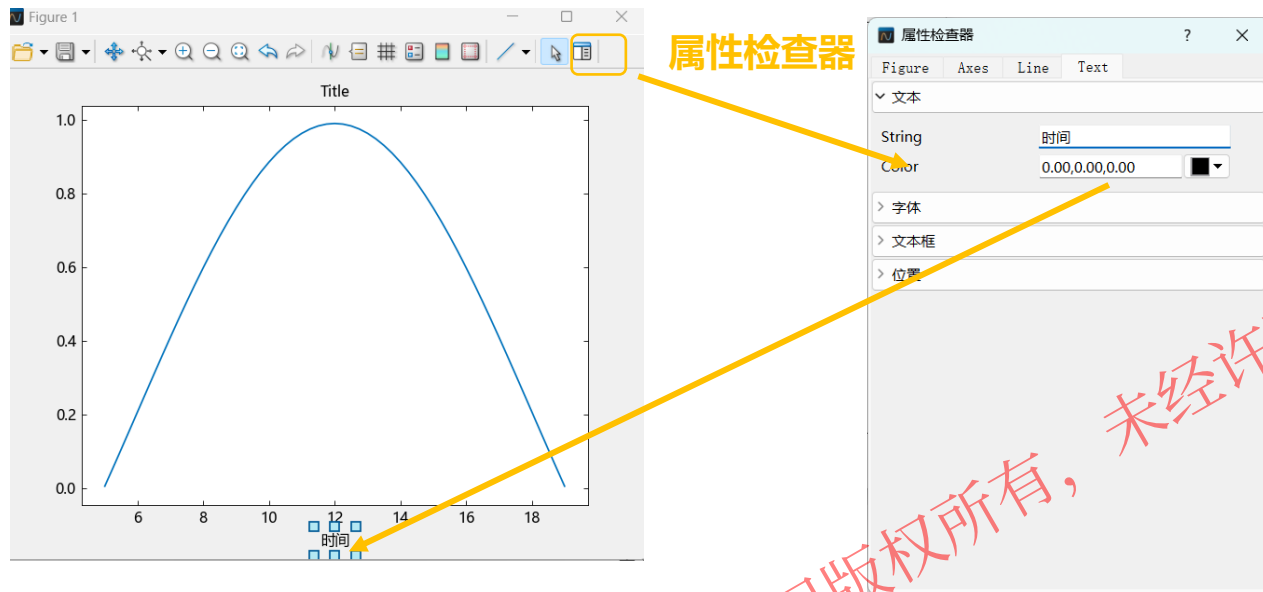


```
using TyPlot
using TyMath

dec = 23.4 # 太阳偏角: 23° 24'
lat = 31 + 30 / 60 # 纬度: 31° 30'
dec = deg2rad(dec) # 将角度转换为弧度
lat = deg2rad(lat)
t = 5:0.25:19 # 默认时间从早上5点开始, 到19点结束
# 计算太阳与电池板之间的夹角的函数
sunangle = sin(dec) * sin(lat) .+ cos(dec) .*
cos(lat) .* cosd.(15 .* (t .- 12))
```

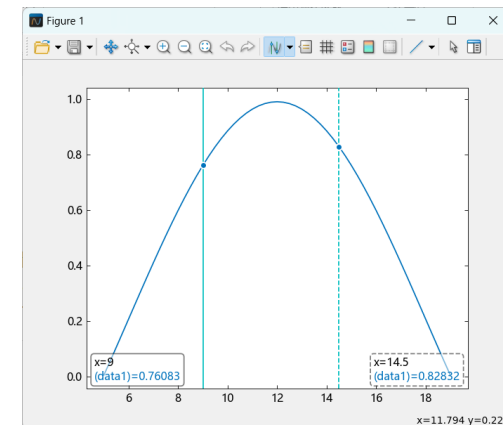
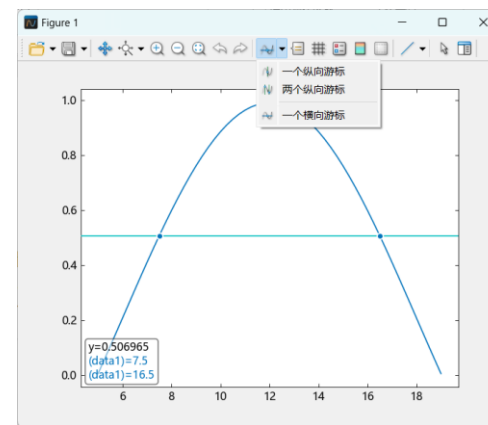
5.1 结果后处理-数据可视化(图形化操作)

可在图形窗口直接配置属性，导出图片



说明:

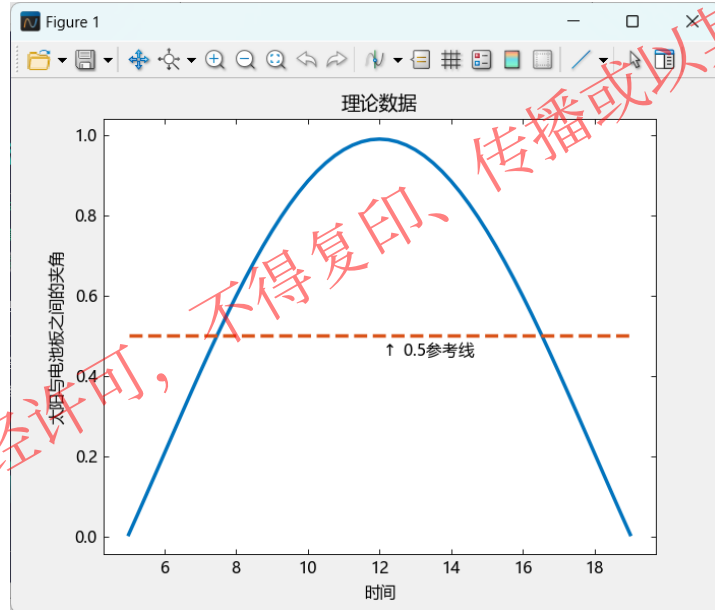
- 支持设置图窗颜色等属性
- 支持设置坐标轴字体、刻度、标尺、网格、标签、框样式、位置等属性
- 支持设置图线颜色、标记、图例等属性
- 支持曲线双 y 轴游标、横向游标等



5.2 结果后处理-数据可视化(脚本)

```
#加载工具箱
using TyPlot
using TyMath
#定义变量
dec = 23.4 # 太阳偏角: 23° 24'
lat = 31 + 30 / 60 # 纬度: 31° 30'
dec = deg2rad(dec) # 将角度转换为弧度
lat = deg2rad(lat)
t = 5:0.25:19 # 默认时间从早上5点开始, 到19点结束
# 计算太阳与电池板之间的夹角的函数
sunangle = sin(dec) * sin(lat) .+ cos(dec) .* cos(lat) .*
cosd.(15 .* (t .- 12))

plot(t, sunangle,linewidth = 2,t,ones(size(t))*0.5,"--") #
绘制夹角与时间的曲线,线宽为2,虚线0.5参考线
xlabel("时间") # 添加X轴标签
ylabel("太阳与电池板之间的夹角") # 添加Y轴标签
title("理论数据") # 添加图标题
text(12, 0.45,raw"$\uparrow$ 0.5参考线")#在(12, 0.45)位置处
插入"↑0.5参考线"
```



title — 给图形加标题

xlabel — 给x轴加标注

ylabel — 给y轴加标注

text — 在图形指定位置加标注

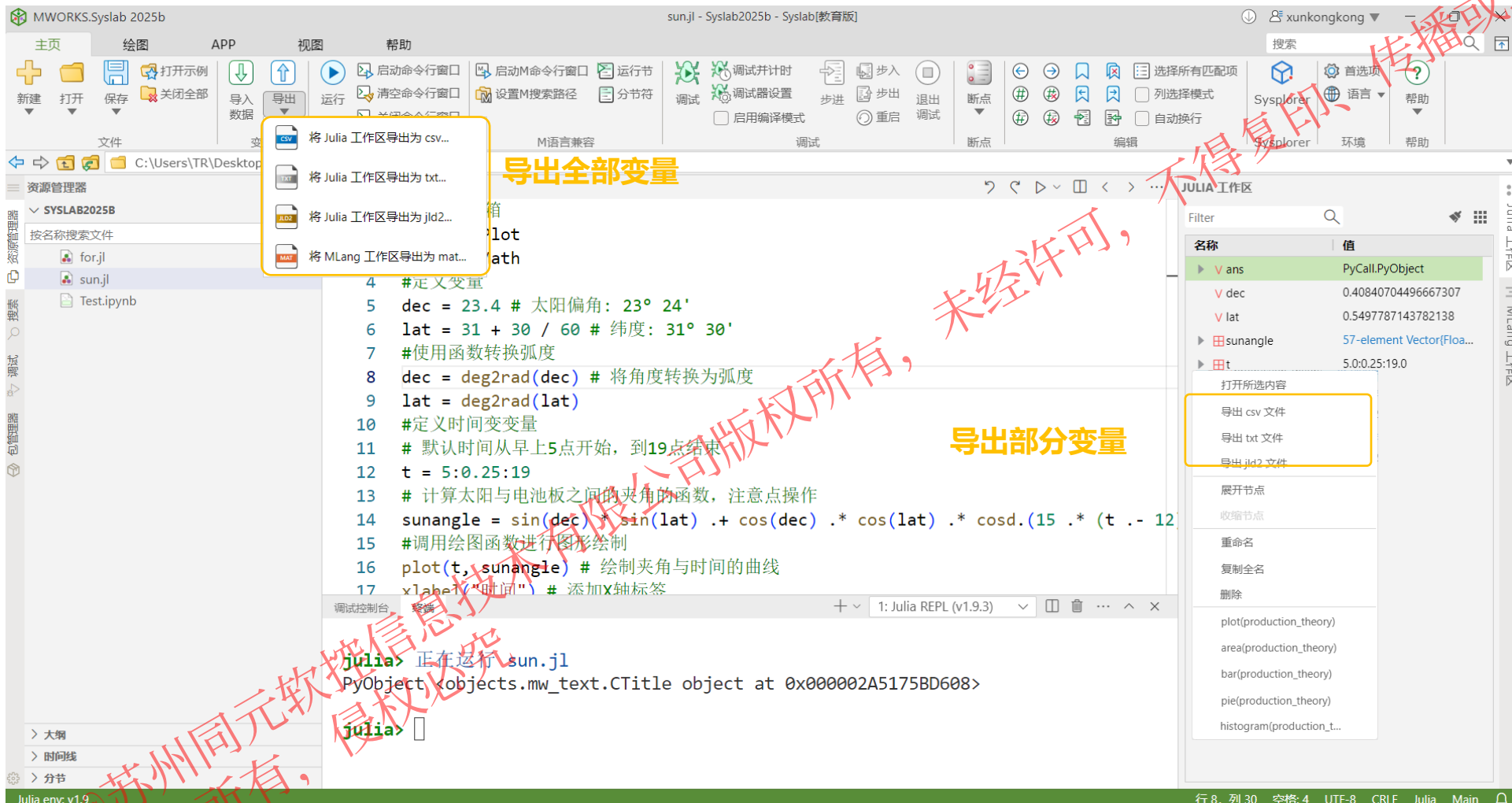
grid ("on/off") — 打开、关闭坐标网格线

legend — 添加图例

axis — 控制坐标轴的刻度

5.3 结果后处理-结果导出

结果导出：所有计算结果和部分计算结果可直接导出为csv、txt、jld2格式，Mlang工作区导出mat格式



说明：

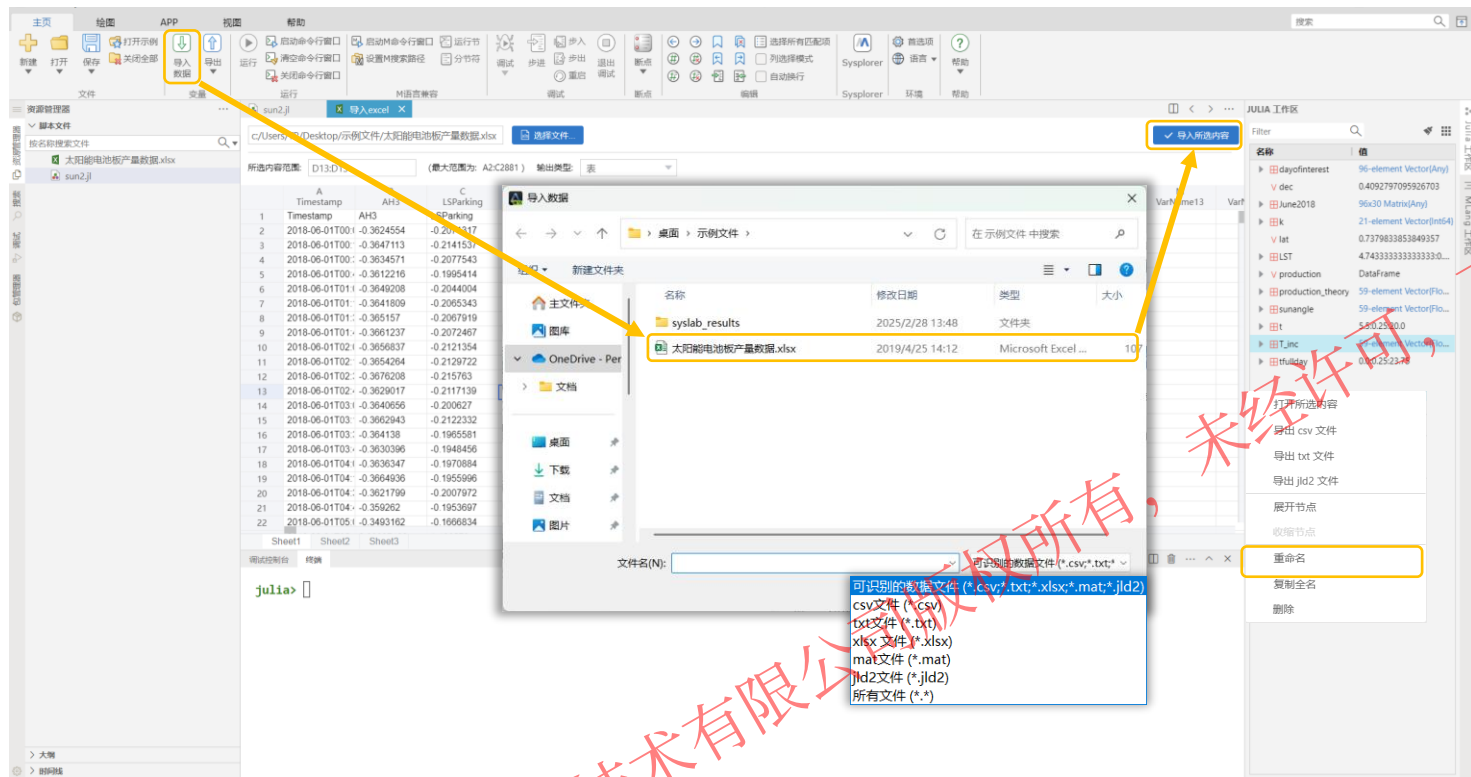
◆ 支持导出文件格式

- CSV
- txt
- jld2

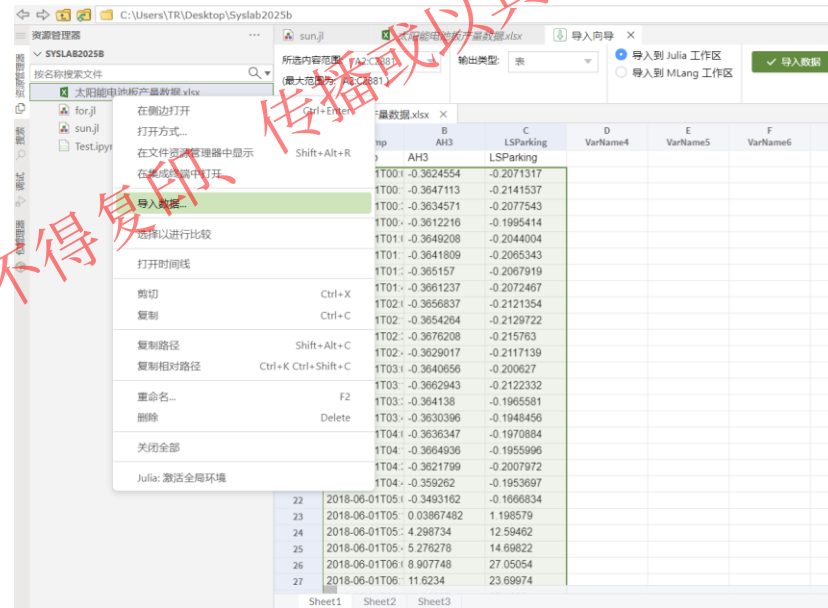
◆ 支持导出全部和部分计算结果

5.4 结果后处理-结果导入

方式一:可将数据文件直接导入至工作空间进行使用



方式二:右键导入、拖拽至工作区导入功能



说明:

◆ 支持导入文件格式

- 文本: txt、csv
- Excel: xlsx
- Mat
- jld2

◆ 工作区查看变量, 可对变量重命名

@苏州同元软控信息技术有限公司
版权所有, 侵权必究

06

PART 06 ➔

入门实践：太阳能电池性能验证

@苏州同元软件信息技术有限公司版权所有，侵权必究

未经许可，不得复印、传播或以其他方式使用

6.1 Syslab一般应用流程



Syslab



理论分析



代码构建



运行调试



结果处理

需求分析、机理分析、
逻辑推导

函数构建、脚本构建
函数调用

Debug调试、代码运行

数据可视化、数据分析、
数据/文件IO

6.2 实例题目

题目：苏州某地安装有太阳能电池板，拟对比**实际产量**与**理论产量**，验证其性能

已知数据：

- ◆ 当地**太阳赤纬角**约为 $23^{\circ} 24'$ ，**纬度**为 $31^{\circ} 30'$
- ◆ **电池板面积**为 270 m^2 ，最大输出为 207 kw
- ◆ 电池板**某月份的产量数据**，提供Excel文件



@苏州同元软控信息技术有限公司版权所有，侵权必究

6.3 理论分析

题目：苏州某地安装有太阳能电池板，拟对比**实际产量**与**理论产量**，验证其性能

实际监测数据

理论计算数据

已知数据：

- ◆ 当地**太阳赤纬角**约为 $23^{\circ} 24'$ ，**纬度**为 $31^{\circ} 30'$
- ◆ **电池板面积**为 270 m^2 ，最大输出为 207 kw
- ◆ 电池板**某月份的产量数据**，提供Excel文件

理论公式：

- ◆ 太阳与电池板之间的夹角公式

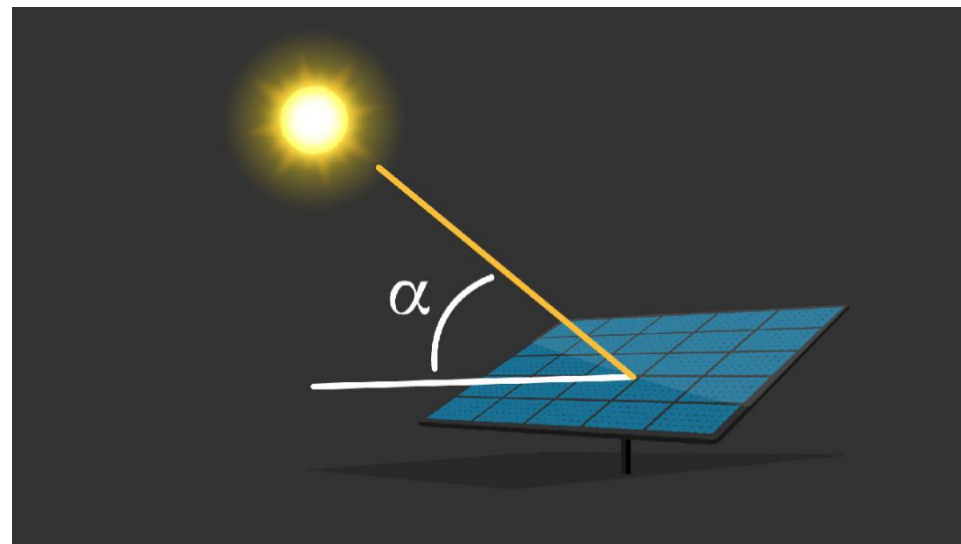
$$\sin(\alpha) = \sin(\underbrace{\delta}_{\text{太阳赤纬角}}) \sin(\underbrace{\varphi}_{\text{纬度}}) + \cos(\underbrace{\delta}_{\text{太阳赤纬角}}) \cos(\underbrace{\varphi}_{\text{纬度}}) \cos(\underbrace{15(t-12)}_{\text{时间}})$$

- ◆ 考虑大气层影响的经验系数公式

$$T_{inc} = 1.4883 \times 0.7^{\sin(\alpha) - 0.678}$$

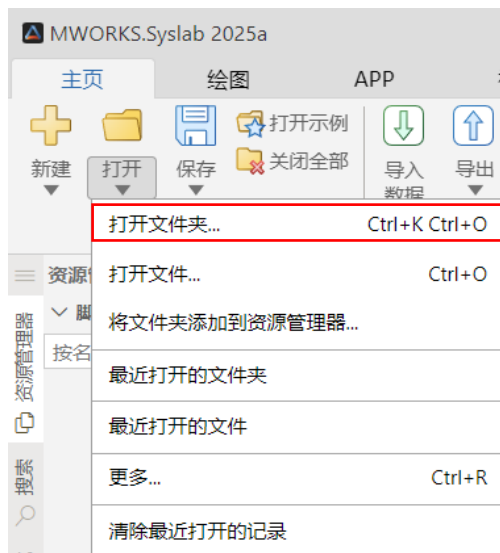
- ◆ 电池板产量公式

$$P = \underbrace{S}_{\text{电池板面积}} \sin(\alpha) T_{inc}$$



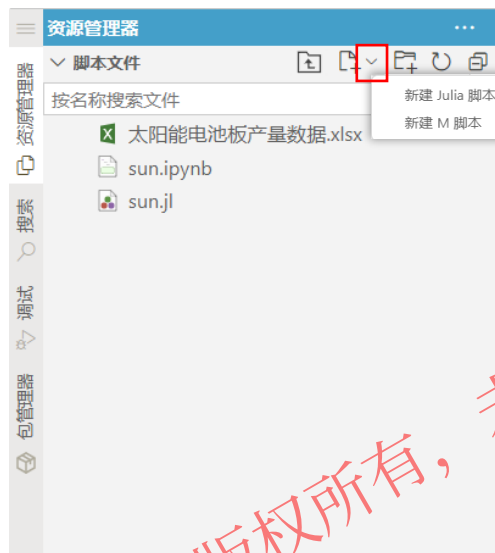
6.4 代码构建

打开相关文件夹作为工作目录，在资源管理器中新建、编辑、管理文件



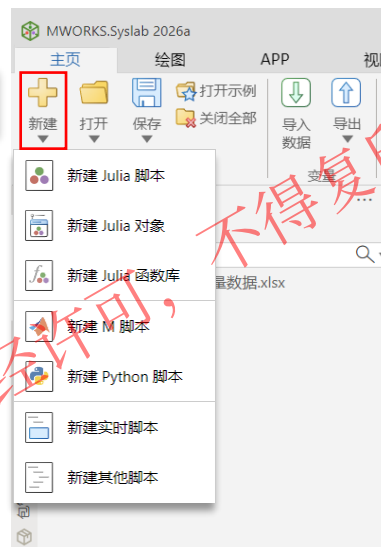
选择工作目录：

主页菜单栏中“打开”文件夹



新建脚本：

支持新建Julia、M、Python等脚本



编辑脚本：

对脚本进行剪切、复制、复制路径、重命名、删除等

@苏州同元软控信息技术有限公司版权所有，侵权必究

6.4 代码构建

已知数据:

- ◆ 当地**太阳赤纬角**约为 $23^{\circ} 24'$, **纬度**为 $31^{\circ} 30'$
- ◆ **电池板面积**为 270 m^2 , 最大输出为 207 kw
- ◆ 电池板**某月份的产量数据**, 提供Excel文件

理论公式:

- ◆ 太阳与电池板之间的夹角公式

$$\sin(\alpha) = \sin(\underbrace{\delta}_{\text{太阳赤纬角}}) \sin(\underbrace{\varphi}_{\text{纬度}}) + \cos(\underbrace{\delta}_{\text{太阳赤纬角}}) \cos(\underbrace{\varphi}_{\text{纬度}}) \cos(15(\underbrace{t}_{\text{时间}} - 12))$$

- ◆ 考虑大气层影响的经验系数公式

$$T_{inc} = 1.4883 \times 0.7^{\sin(\alpha)^{-0.678}}$$

- ◆ 电池板产量公式

$$P = \underbrace{S}_{\text{电池板面积}} \sin(\alpha) T_{inc}$$

```
dec = 23 + 24 / 60 # 太阳赤纬角: 23° 24'
lat = 31 + 30 / 60 # 纬度: 31° 30'
dec = deg2rad(dec) # 将角度转换为弧度
lat = deg2rad(lat)
t = 5:0.25:19 # 时间从早上5点开始, 到19点结束
# 定义计算太阳与电池板之间的夹角的函数
sunangle = sin(dec) * sin(lat) .+ cos(dec) .*
cos(lat) .* cosd.(15 .* (t .- 12))
plot(t, sunangle) # 绘制太阳与电池板之间的夹角与时间的曲线
T_inc = 1.4883 * 0.7 .^ (sunangle .^ -0.678) # 考虑大气层影响
production_theory = 270 * T_inc .* sunangle # 计算电池板理论产量
k = find(production_theory .> 207) # 考虑电池板最大输出为207kw
production_theory[k] .= 207
plot(t, production_theory) # 绘制理论产量与时间的曲线
xlabel("时间") # 添加X轴标签
ylabel("产量(kW)") # 添加Y轴标签
title("理论数据") # 添加图标题
```

计算理论产量

6.4 代码构建

已知数据:

- ◆ 当地**太阳赤纬角**约为 $23^{\circ} 24'$, **纬度**为 $31^{\circ} 30'$
- ◆ **电池板面积**为 270 m^2 , 最大输出为 207 kw
- ◆ 电池板**某月份的产量数据**, 提供Excel文件

理论公式:

- ◆ 太阳与电池板之间的夹角公式

$$\sin(\alpha) = \sin(\underbrace{\delta}_{\text{太阳赤纬角}}) \sin(\underbrace{\varphi}_{\text{纬度}}) + \cos(\underbrace{\delta}_{\text{太阳赤纬角}}) \cos(\underbrace{\varphi}_{\text{纬度}}) \cos(15(\underbrace{t}_{\text{时间}} - 12))$$

- ◆ 考虑大气层影响的经验系数公式

$$T_{inc} = 1.4883 \times 0.7^{\sin(\alpha)^{-0.678}}$$

- ◆ 电池板产量公式

$$P = \underbrace{S}_{\text{电池板面积}} \sin(\alpha) T_{inc}$$

```
production = readtable("太阳能电池板产量数据.xlsx") # 读取实际监测数据
```

```
plot(production.Timestamp, production.AH3) # 绘制电池板整月实际产量与时间的曲线
```

```
June2018 = reshape(production.AH3, 96, 30) # 将产量的向量数据转换为矩阵数据, 每列为一天内的数据
```

```
dayofinterest = June2018[:, 26] # 索引26号的数据
```

```
tfullday = 0:0.25:23.75 # 定义一天内的时间
```

```
plot(tfullday, dayofinterest, "-.")
```

```
plot(tfullday, dayofinterest, "-.", t, production_theory) # 对比绘制26号的电池板实际产量和理论产量与时间的曲线
```

```
xlabel("时间")
```

```
ylabel("产量(kW)")
```

```
legend("实际数据", "理论数据")
```

对比实际和理论产量

6.5 运行调试

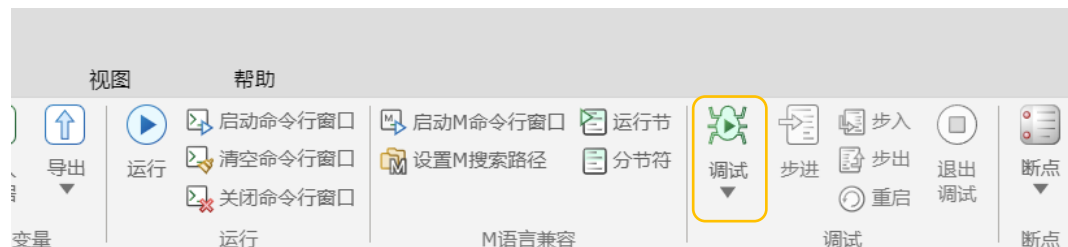
```
dec = 23.4          # 太阳赤纬角: 23° 24'
lat = 31 + 30 / 60   # 纬度: 31° 30'
dec = deg2rad(dec)   # 将角度转换为弧度
lat = deg2rad(lat)
t = 5:0.25:19        # 默认时间从早上5点开始, 到19点结束
# 定义计算太阳与电池板之间的夹角的函数
sunangle = sin(dec) * sin(lat) .+ cos(dec) .*
cos(lat) .* cosd.(15 .* (t .- 12))
plot(t, sunangle) # 绘制太阳与电池板之间的夹角与时间的曲线
T_inc = 1.4883 * 0.7 .^ (sunangle .^ -0.678) # 考虑大气层影响
production_theory = 270 * T_inc .* sunangle # 计算电池板理论产量
k = find(production_theory .> 207) # 考虑电池板最大输出为207kw
production_theory[k] .= 207
plot(t, production_theory) # 绘制理论产量与时间的曲线
xlabel("时间") # 添加x轴标签
ylabel("产量(kW)") # 添加y轴标签
title("理论数据") # 添加图标题
```

代码运行:



- 点击主页菜单栏中"运行", 运行整个脚本文件
- 运行脚本部分代码: **Ctrl / Shift + Enter**
- Syslab 2025a 支持**设置分节符, 运行节**

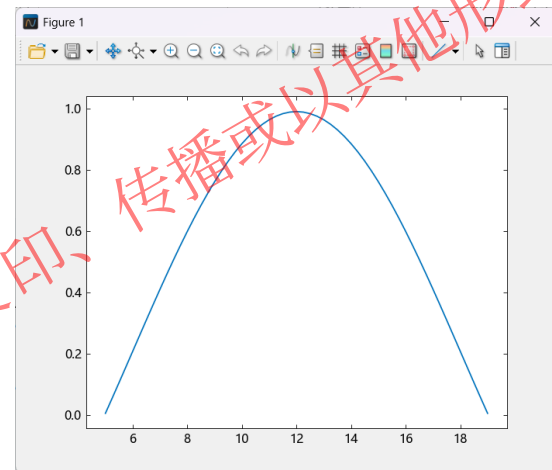
代码调试:



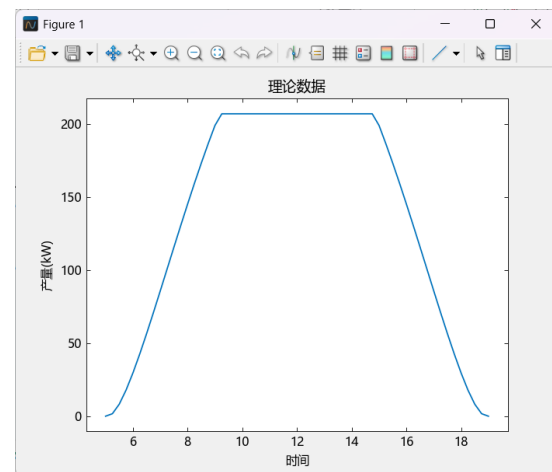
- 在脚本打上断点
- 点击主页菜单栏中"调试"或F5进入DeBug模式

6.5 运行调试

```
dec = 23.4          # 太阳赤纬角: 23° 24'
lat = 31 + 30 / 60  # 纬度: 31° 30'
dec = deg2rad(dec)  # 将角度转换为弧度
lat = deg2rad(lat)
t = 5:0.25:19       # 默认时间从早上5点开始, 到19点结束
# 定义计算太阳与电池板之间的夹角的函数
sunangle = sin(dec) * sin(lat) .+ cos(dec) .*
cos(lat) .* cosd.(15 .* (t .- 12))
plot(t, sunangle) # 绘制太阳与电池板之间的夹角与时间的曲线
T_inc = 1.4883 * 0.7 .^ (sunangle .^ -0.678) # 考虑大气层影响
production_theory = 270 * T_inc .* sunangle # 计算电池板理论产量
k = find(production_theory .> 207) # 考虑电池板最大输出为207kw
production_theory[k] .= 207
plot(t, production_theory) # 绘制理论产量与时间的曲线
xlabel("时间") # 添加x轴标签
ylabel("产量(kW)") # 添加y轴标签
title("理论数据") # 添加图标题
```



太阳与电池板之间的夹角与时间曲线



电池板理论产量与时间曲线

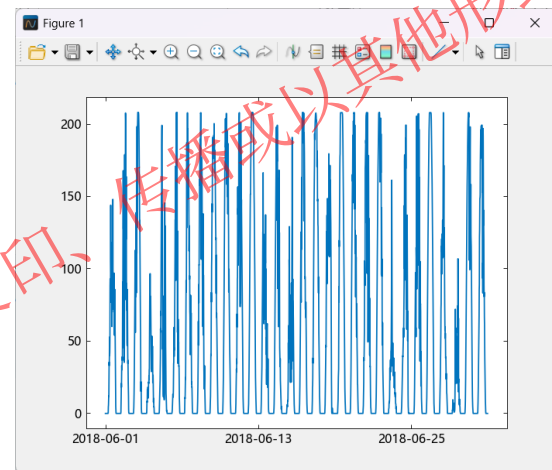
6.5 查看结果

```
production = readtable("太阳能电池板产量数据.xlsx")
# 读取实际监测数据, 函数实现
plot(production.Timestamp, production.AH3) # 绘制电池板整月实际产量与时间的曲线

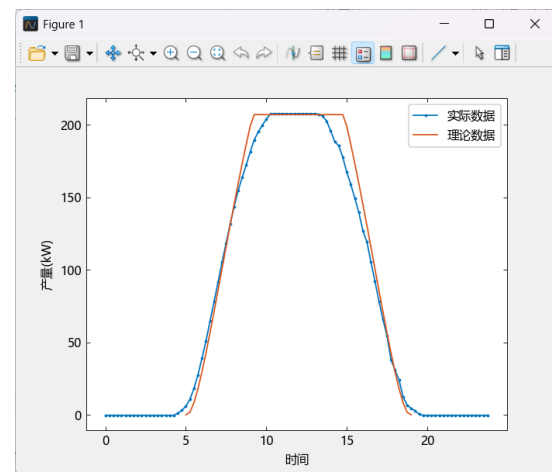
June2018 = reshape(production.AH3, 96, 30) # 将产量的向量数据转换为矩阵数据, 每列为一天内的数据
dayofinterest = June2018[:, 26] # 索引26号的数据
tfullday = 0:0.25:23.75 # 定义一天内的时间

plot(tfullday, dayofinterest, ".-")
plot(tfullday, dayofinterest, ".-", t, production_theory) # 对比绘制26号的电池板实际产量和理论产量与时间的曲线
xlabel("时间")
ylabel("产量(kW)")
legend("实际数据", "理论数据")
```

结论: 太阳能电池板实际产量与理论产量较相近, 性能符合预期



电池板整月实际产量
与时间曲线



电池板实际产量和理论产量
与时间曲线

Thanks.



©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
侵权必究